



Tema 2. Representación de la información en computadores

Grupo A:
Fernando Rojas
frojas@ugr.es

Grupo B:
Samuel Romero
sromero@ugr.es

Grupo C:
Juan Luis Jiménez
juanlu@ugr.es

Para estudiar esta diapositiva

Este tema estudia cómo representar información mediante bits. La misma secuencia puede significar un número, un carácter, una muestra de sonido o parte de una imagen: siempre necesitamos conocer la convención utilizada.

¿Cómo contamos usando solo dos estados?



Para estudiar esta diapositiva

Dos estados bastan para construir muchas combinaciones. Con n posiciones binarias se obtienen 2^n patrones; la clave no es disponer de muchos símbolos, sino combinar y ordenar unos pocos.

Tema 2 · Nuestro recorrido

L2.0 Sistemas de numeración usuales en informática

L2.1 Nociones básicas sobre representación de la información

L2.2 Representación de textos

L2.3 Representación de sonidos

L2.4 Representación de imágenes

L2.5 Representación de vídeo

L2.6 Representación de números enteros

L2.7 Representación de números reales

Para estudiar esta diapositiva

El recorrido conecta tres ideas: codificar información, calcular cuánto espacio ocupa e interpretar correctamente los bits. Las primeras lecciones proporcionan herramientas que volverán a aparecer al estudiar enteros y reales.

L2.0

Sistemas de numeración usuales en informática

01 Posición 02 Binario 03 Hexadecimal 04 Aritmética

Para estudiar esta diapositiva

En un sistema posicional, el valor de cada cifra depende de la base y de su posición. Aprenderemos a conservar el valor mientras cambiamos su escritura y a operar en binario.

L2.0 · De las cifras a las operaciones

01 Posición

El valor de una cifra depende de dónde está.

02 Binario

Leer y escribir números usando 0 y 1.

03 Hexadecimal

Abreviar grupos de cuatro bits.

04 Aritmética

Sumar, restar y multiplicar en base 2.

Para estudiar esta diapositiva

Primero identifica los pesos de las posiciones; después utiliza esos pesos para convertir y comprobar operaciones. Hexadecimal no es otra clase de dato: es una escritura compacta de grupos de bits.

01 · Posición

Numeración indo-arábica → Un ejemplo conocido: **4567,28**

Cada cifra aporta un valor según la posición que ocupa.

4	5	6	7	,	2	8
POSICIÓN millares	POSICIÓN centenas	POSICIÓN decenas	POSICIÓN unidades		POSICIÓN décimas	POSICIÓN centésimas
APORTA 4000	APORTA 500	APORTA 60	APORTA 7		APORTA 0,2	APORTA 0,08

$$4567,28 = 4000 + 500 + 60 + 7 + 0,2 + 0,08$$

Para estudiar esta diapositiva

En 4567,28, el 4 aporta 4×10^3 y el 8 aporta 8×10^{-2} . La coma separa potencias no negativas de potencias negativas; la misma regla se aplica en otras bases.

Cifra × peso: la idea de un sistema posicional

Cifra	4	5	6	7,	2	8
Posición	3	2	1	0	-1	-2
Peso	10^3	10^2	10^1	10^0	10^{-1}	10^{-2}

- En base **b**, cada posición pesa **b** veces la de su derecha.
- Las cifras disponibles van de **0** a **b - 1**.

$$\text{Valor} = \dots + a_2 \cdot b^2 + a_1 \cdot b^1 + a_0 \cdot b^0 + a_{-1} \cdot b^{-1} + \dots$$

Para estudiar esta diapositiva

La fórmula resume una suma de aportaciones independientes. Para comprobar una conversión, multiplica cada cifra por la potencia de la base que le corresponde y suma los resultados.

Si cambiamos de base, cambia la escritura, no el valor

Decimal	Binario	Hexadecimal
BASE 10	BASE 2	BASE 16
SÍMBOLOS DISPONIBLES 0–9	SÍMBOLOS DISPONIBLES 0, 1	SÍMBOLOS DISPONIBLES 0–9 y A–F A = 10, B = 11, ..., F = 15
Mismo valor, distintas representaciones $(10)_{10} = (1010)_2 = (A)_{16}$		

Para estudiar esta diapositiva

Diez, 1010_2 y A_{16} pueden representar la misma cantidad. Los subíndices evitan confundir la escritura con el valor; cambiar de base no modifica la cantidad representada.

02 · Binario — Cada posición a la izquierda pesa el doble

Solo usamos dos cifras: **0** y **1**

Posición 3	Posición 2	Posición 1	Posición 0
2^3	2^2	2^1	2^0
8	4	2	1
Peso	Peso	Peso	Peso

Un **1** activa el peso de su posición; un **0** no aporta valor.

Para estudiar esta diapositiva

Los pesos binarios crecen como 1, 2, 4, 8... al avanzar a la izquierda. Una cifra 0 omite ese peso y una cifra 1 lo incorpora a la suma.

¿Qué valor tiene $(00001001)_2$?

Interpreta este patrón como un número entero sin signo.

0

0

0

0

1

0

0

1

Pista: Cada posición activada con un **1** aporta el peso correspondiente a su potencia de 2 ($2^3 = 8$ y $2^0 = 1$).

Para estudiar esta diapositiva

Empieza por el bit de la derecha, cuyo peso es 1. Solo debes sumar los pesos de las posiciones con un 1; los ceros iniciales no alteran el valor sin signo.

¿Qué valor tiene $(00001001)_2$?

Interpreta este patrón como un número entero sin signo.

Posición 7	Posición 6	Posición 5	Posición 4	Posición 3	Posición 2	Posición 1	Posición 0
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
0	0	0	0	1	0	0	1

Posición 0: su peso es $2^0 = 1$.

Suma los pesos activados por un **1** (Posición 3 = 8, Posición 0 = 1):

$$8 + 1 = 9 \rightarrow (00001001)_2 = (9)_{10}$$

Para estudiar esta diapositiva

Los dos unos activan los pesos 8 y 1. Por eso el resultado es 9; no se interpreta la cadena como un número decimal escrito con ceros y unos.

¿Qué valor tiene $(00001001)_2$?

Interpreta este patrón como un número entero sin signo.

Posición 7	Posición 6	Posición 5	Posición 4	Posición 3	Posición 2	Posición 1	Posición 0
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
0	0	0	0	1	0	0	1

Posición 0: su peso es $2^0 = 1$.
Suma los pesos activados por un **1** (Posición 3 = 8, Posición 0 = 1):
 $8 + 1 = 9 \rightarrow (00001001)_2 = (9)_{10}$



Para estudiar esta diapositiva

Comprueba la respuesta recorriendo las posiciones de derecha a izquierda: 1, 2, 4 y 8. Esta comprobación por pesos es independiente del método utilizado para obtener el patrón.

Ahora al revés: de decimal a binario

Escribe el número decimal 74 en base 2.

$$(74)_{10} = (?)_2$$

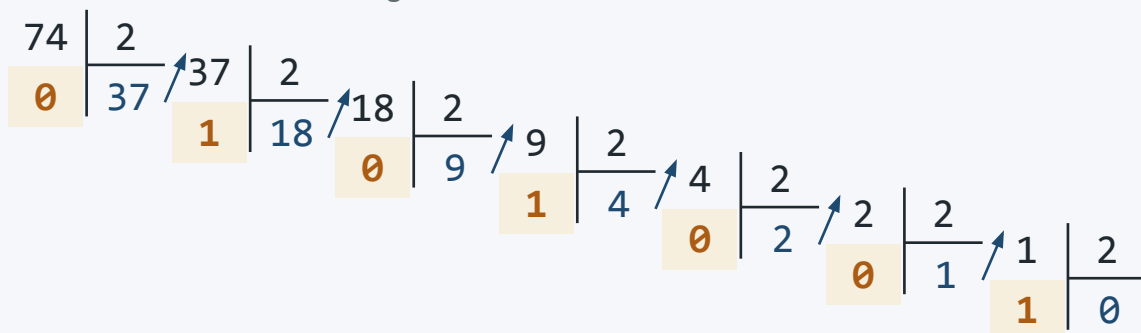
Busca las cifras binarias sin cambiar el valor del número.

Para estudiar esta diapositiva

Para convertir 74, busca una combinación de potencias de 2 cuya suma sea 74. Otra opción equivalente consiste en dividir sucesivamente entre 2 y conservar los restos.

Divisiones sucesivas entre 2

El cociente continúa en la siguiente división.



Leemos los restos desde la última división: **1 0 0 1 0 1 0**

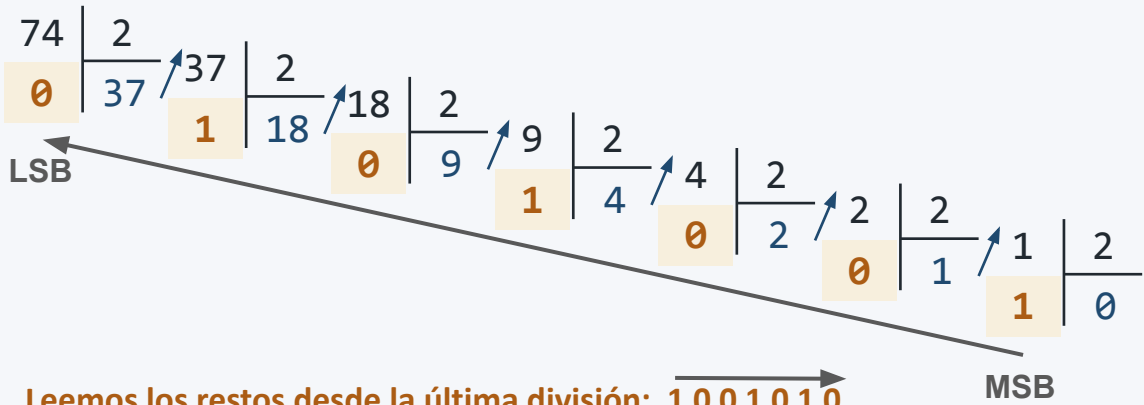
$$(74)_{10} = (1001010)_2 \quad \cdot \quad 64 + 8 + 2 = 74$$

Para estudiar esta diapositiva

Cada resto indica una cifra binaria, empezando por la menos significativa. Por eso los restos se leen en orden inverso al de obtención: $1001010_2 = 64 + 8 + 2$.

Divisiones sucesivas entre 2

El cociente continúa en la siguiente división.



$$(74)_{10} = (1001010)_2 \quad \cdot \quad 64 + 8 + 2 = 74$$

Para estudiar esta diapositiva

LSB significa bit menos significativo y MSB, bit más significativo. Se refieren al peso posicional, no al orden temporal en que se hayan calculado los bits.

¿Y si el número tiene parte fraccionaria?

Convierte 0,423 a binario y conserva seis cifras después de la coma.

$$(0,423)_{10} \approx (0,??????)_2$$

Los pesos ahora son $2^{-1}=1/2$, $2^{-2}=1/4$, $2^{-3}=1/8$, ...

Para estudiar esta diapositiva

La parte fraccionaria se convierte mediante multiplicaciones por 2. Al limitar la respuesta a seis posiciones puede aparecer una aproximación: conviene distinguir el signo igual del signo aproximadamente igual.

Multiplicamos la fracción y recogemos la parte entera

Cada fracción continúa en la multiplicación siguiente.

$$\begin{array}{r}
 0,423 \\
 \times 2 \\
 \hline
 0,846
 \end{array}
 \begin{array}{r}
 0,846 \\
 \times 2 \\
 \hline
 1,692
 \end{array}
 \begin{array}{r}
 0,692 \\
 \times 2 \\
 \hline
 1,384
 \end{array}
 \begin{array}{r}
 0,384 \\
 \times 2 \\
 \hline
 0,768
 \end{array}
 \begin{array}{r}
 0,768 \\
 \times 2 \\
 \hline
 1,536
 \end{array}
 \begin{array}{r}
 0,536 \\
 \times 2 \\
 \hline
 1,072
 \end{array}$$

Partes enteras, en este orden: $0 \rightarrow 1 \rightarrow 1 \rightarrow 0 \rightarrow 1 \rightarrow 1$

$(0,423)_{10} \approx (0,011011)_2 = 0,421875$ (truncado a seis bits)

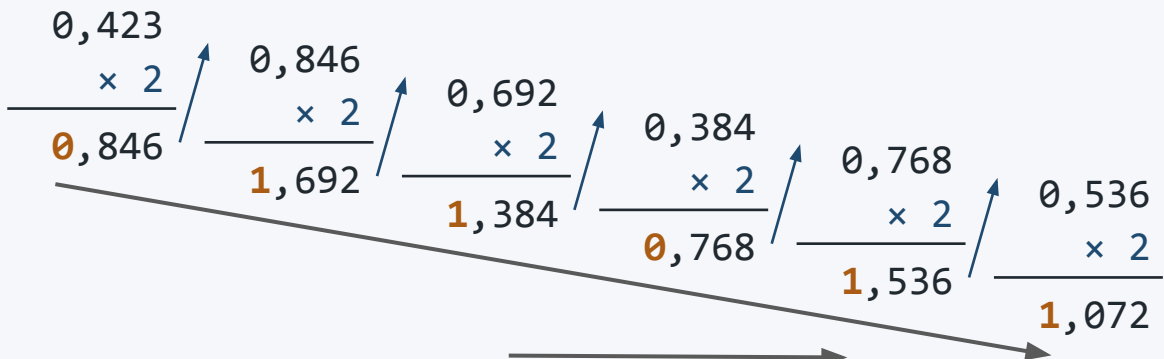
12 / 32

Para estudiar esta diapositiva

En cada multiplicación se recoge la parte entera y se continúa únicamente con la fracción restante. Aquí $0,011011_2$ vale $0,421875$; el truncamiento produce una diferencia de $0,001125$ respecto de $0,423$.

Multiplicamos la fracción y recogemos la parte entera

Cada fracción continúa en la multiplicación siguiente.



Partes enteras, en este orden: 0 → 1 → 1 → 0 → 1 → 1

$(0,423)_{10} \approx (0,011011)_2 = 0,421875$ (truncado a seis bits)

Para estudiar esta diapositiva

Los bits fraccionarios se leen en el orden en que aparecen, a diferencia de los restos de las divisiones de la parte entera. Se ha truncado a seis bits, no aplicado un redondeo al más cercano.

03 · Hexadecimal

Una forma breve de escribir bits

Cuatro bits ($2^4 = 16$ combinaciones) se representan exactamente con **una sola cifra hexadecimal**.

Binario	0000	0001	...	1001	1010	...	1111
Hexadecimal	0	1	...	9	A	...	F



Conclusión: La agrupación en bloques de 4 bits reduce drásticamente la longitud de la escritura.

Para estudiar esta diapositiva

La equivalencia exacta se debe a que $16=2^4$. Cada grupo de cuatro bits tiene una correspondencia única con una cifra de 0 a F; esto permite abreviar patrones sin perder información.

De hexadecimal a decimal: cifra × peso

Interpretamos cada cifra según su posición. En hexadecimal, A vale 10.

$$\begin{array}{c} \mathbf{2} \\ \downarrow \\ 2 \times 256 \\ \text{peso } 16^2 \end{array}$$

$$\begin{array}{c} \mathbf{A} \\ \downarrow \\ 10 \times 16 \\ \text{peso } 16^1 \end{array}$$

$$\begin{array}{c} \mathbf{6} \\ \downarrow \\ 6 \times 1 \\ \text{peso } 16^0 \end{array}$$

Cada posición pesa 16 veces la situada a su derecha.

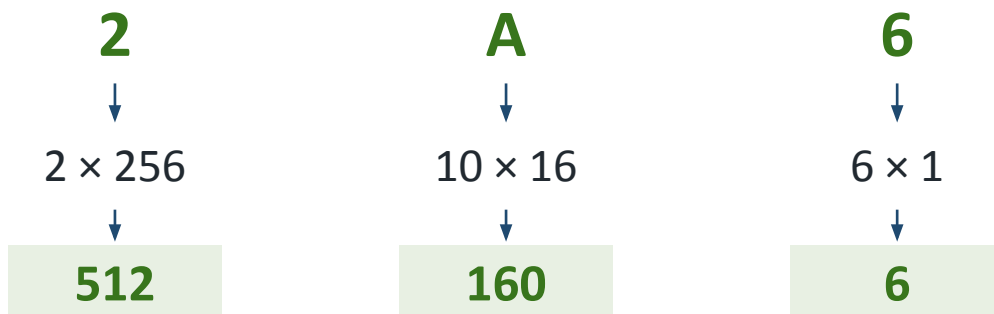
15 / 32

Para estudiar esta diapositiva

Los pesos de $2A6_{16}$ son 16^2 , 16^1 y 16^0 . Recuerda sustituir A por el valor decimal 10 antes de hacer las multiplicaciones.

De hexadecimal a decimal: cifra × peso

Interpretamos cada cifra según su posición. En hexadecimal, A vale 10.



$$(2A6)_{16} = 512 + 160 + 6 = (678)_{10}$$

Para estudiar esta diapositiva

La suma $512+160+6$ comprueba que $2A6_{16}$ vale 678. No se deben concatenar los resultados parciales: se suman porque son aportaciones al mismo valor.

De decimal a hexadecimal

Divisiones sucesivas entre 16

$$\begin{array}{r|l} 678 & 16 \\ \hline 6 & 42 \end{array} \longrightarrow \begin{array}{r|l} 42 & 16 \\ \hline 10 & 2 \end{array} \longrightarrow \begin{array}{r|l} 2 & 16 \\ \hline 2 & 0 \end{array}$$

← Restos: 2, 10, 6 → cifras: 2, A, 6

$$(678)_{10} = (2A6)_{16}$$

Para estudiar esta diapositiva

Los restos de las divisiones entre 16 se convierten en cifras hexadecimales. El resto 10 se escribe A; leídos desde la última división, los restos forman 2A6.

De binario a hexadecimal: agrupamos cuatro bits

Partimos de la coma: hacia la izquierda y hacia la derecha.

1 1 1 0 0 0 1 0 1 1 0 0 1 1 , 0 0 0 1 1 1

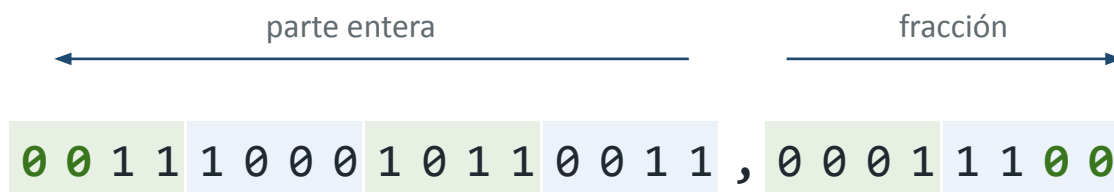
¿Por dónde empezamos a formar los grupos?

Para estudiar esta diapositiva

La coma es el punto de referencia para agrupar. En la parte entera se agrupa hacia la izquierda y, en la fraccionaria, hacia la derecha; nunca se atraviesa la coma con un grupo.

De binario a hexadecimal: agrupamos cuatro bits

Partimos de la coma: hacia la izquierda y hacia la derecha.



Completamos con ceros los grupos de los extremos, sin cambiar el valor.

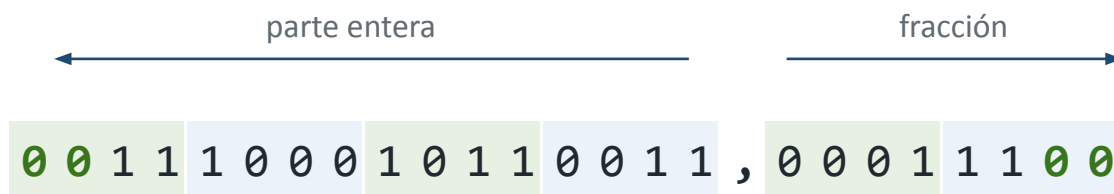
Cada grupo de cuatro bits se convertirá en una cifra hexadecimal.

Para estudiar esta diapositiva

Completar con ceros a la izquierda del entero o a la derecha de la fracción conserva el valor. Añadir ceros en el lado opuesto sí puede cambiarlo.

De binario a hexadecimal: agrupamos cuatro bits

Partimos de la coma: hacia la izquierda y hacia la derecha.



Completamos con ceros los grupos de los extremos, sin cambiar el valor.

↓ ↓ ↓ ↓ ↓ ↓

3 8 B 3 , 1 C

Resultado: (38B3,1C)₁₆

20 / 32

Para estudiar esta diapositiva

Sustituye cada grupo de cuatro bits por su cifra hexadecimal manteniendo la coma en su lugar. El resultado 38B3,1C₁₆ abrevia exactamente la misma secuencia binaria.

De hexadecimal a binario: cuatro bits por cifra

Sustituimos cada cifra hexadecimal por su grupo de cuatro bits.

2

A

6

Una cifra → cuatro bits, incluidos los ceros iniciales del grupo.

21 / 32

Para estudiar esta diapositiva

Al desplegar hexadecimal, cada cifra ocupa cuatro bits, incluso si empieza por ceros. Mantenerlos evita desplazar las posiciones de los grupos siguientes.

De hexadecimal a binario: cuatro bits por cifra

Sustituimos cada cifra hexadecimal por su grupo de cuatro bits.

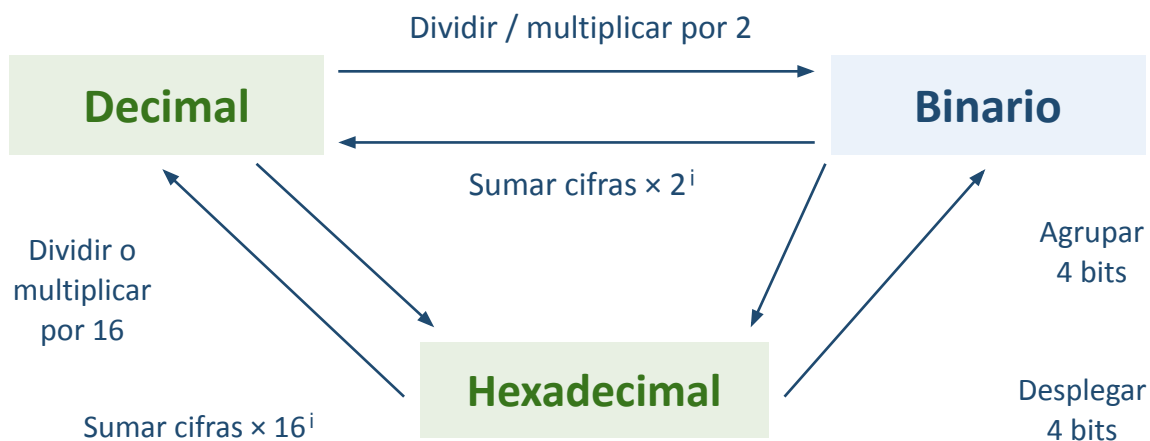


$$(2A6)_{16} = (0010\ 1010\ 0110)_2$$

Para estudiar esta diapositiva

2, A y 6 se convierten respectivamente en 0010, 1010 y 0110. La agrupación visual ayuda a comprobar la conversión; los espacios no son bits almacenados.

Podemos convertir entre todas las bases



Base $b \rightarrow$ decimal $\rightarrow$ base c : un camino válido para cualquier par de bases.

23 / 32

Para estudiar esta diapositiva

Pasar por decimal funciona como método general, pero entre binario y hexadecimal basta agrupar o desplegar bits. Elegir el camino corto reduce operaciones y errores.

04 · Aritmética

¿Cómo operamos con estas cifras?

Las reglas son las mismas que en decimal,
pero agrupamos de dos en dos.

$$1 + 1 = 10_2$$

Dos unidades
forman una de peso 2.

Primero sumamos; después usamos la idea de préstamo.

Para estudiar esta diapositiva

La aritmética posicional sigue la misma lógica en cualquier base. En binario, dos unidades de una columna se convierten en una unidad de la columna siguiente.

Sumamos en base 2

Operamos con números binarios sin signo.

$$\begin{array}{r} 1 \ 1 \ 0 \ 1 \ 0 \\ + \ 1 \ 0 \ 1 \ 1 \ 1 \\ \hline \end{array}$$

Para estudiar esta diapositiva

Alinea primero las columnas por su peso. En cada una suma los bits de los operandos y el posible acarreo recibido de la columna anterior.

Sumamos en base 2

Operamos con números binarios sin signo.

$$\begin{array}{rcccccc} & & & & & 1 & \\ & & & & & & \\ & & 1 & 1 & 0 & 1 & 0 & \\ + & 1 & 0 & 1 & 1 & 1 & & \\ \hline & & & & & 0 & 1 & \end{array}$$

De derecha a izquierda:

$$0 + 1 = 1$$

$$1 + 1 = 10_2$$

Escribimos 0 y llevamos 1.

Para estudiar esta diapositiva

Cuando la suma local vale 2, se escribe 0 y se lleva 1. Ese acarreo representa una unidad de la posición siguiente, cuyo peso es el doble.

Sumamos en base 2

Operamos con números binarios sin signo.

$$\begin{array}{r}
 \\
 \\
 \\
 + \\
 \hline
 1 1 0 0 1
 \end{array}$$

Último acarreo → nueva cifra.

Seguimos con los
acarreos:

$$0 + 1 + 1 = 10_2$$

$$1 + 0 + 1 = 10_2$$

$$1 + 1 + 1 = 11_2$$

$$(11010)_2 + (10111)_2 = (110001)_2 \quad \cdot \quad 26 + 23 = 49$$

Para estudiar esta diapositiva

El acarreo final añade una posición si no se impone un ancho fijo. La comprobación decimal, $26+23=49$, permite verificar toda la suma de manera independiente.

Restamos con préstamos

Operamos con números binarios sin signo.

$$\begin{array}{r} 1\ 0\ 1\ 0\ 1 \\ -\ 0\ 1\ 0\ 1\ 0 \\ \hline \end{array}$$

Para estudiar esta diapositiva

En una resta binaria, pedir prestado a la izquierda aporta dos unidades de la columna actual. Antes de operar, alinea los operandos y completa con ceros si hace falta.

Restamos con préstamos

Operamos con números binarios sin signo.

$$\begin{array}{rcccccc} & & & 0 & 2 & & \\ & & 1 & 0 & 1 & 0 & 1 \\ - & 0 & 1 & 0 & 1 & 0 & \\ \hline & & & & 1 & 1 & \end{array}$$

En la posición de peso 2:
0 – 1 necesita préstamo.
Tomamos 1 de peso 4,
que equivale a 2 de peso 2.

Para estudiar esta diapositiva

El préstamo conserva el valor total: se reduce una unidad del peso superior y se añaden dos del peso inferior. Sigue las modificaciones de las cifras para no reutilizar un bit ya prestado.

Restamos con préstamos

Operamos con números binarios sin signo.

$$\begin{array}{r} 0 2 0 2 \\ 1 0 1 0 1 \\ - 0 1 0 1 0 \\ \hline 0 1 0 1 1 \end{array}$$

Después, en el peso 8:

0 - 1 requiere otro

préstamo.

1 de peso 16 pasa a ser

2 de peso 8.

$$(10101)_2 - (01010)_2 = (01011)_2 \quad \cdot \quad 21 - 10 = 11$$

Para estudiar esta diapositiva

La comprobación decimal es $21 - 10 = 11$. Revisa los préstamos por columnas antes de interpretar el resultado final 01011_2 .

Producto aritmético de números binarios

Reglas del producto

$$0 \times 0 = 0$$

$$0 \times 1 = 0$$

$$1 \times 0 = 0$$

$$1 \times 1 = 1$$

Multiplicación en columna

$$\begin{array}{r}
 10110 \\
 \times 101 \\
 \hline
 10110 \\
 00000 \\
 10110 \\
 \hline
 \end{array}$$

Resultado

1 1 0 1 1 1 0

Para estudiar esta diapositiva

Cada 1 del multiplicador genera una copia desplazada del multiplicando; cada 0 genera un producto parcial nulo. Finalmente se suman los productos parciales con sus pesos correspondientes.

De los números a los patrones de bits

¿Cuánto vale $(1010)_2$? ¿Cómo se escribe en hexadecimal?

¿Por qué una cifra hexadecimal equivale a cuatro bits?

¿Qué cambia al desplazar un entero binario a la izquierda?

L2.1 → Un patrón de bits necesita una interpretación.

32 / 32

Para estudiar esta diapositiva

Como autoevaluación, 1010_2 vale 10 y se escribe A_{16} . Desplazar un entero sin signo a la izquierda equivale a multiplicar por 2 si se dispone de espacio suficiente y no se pierden bits.

Tema 2 · Nuestro recorrido

L2.0 Sistemas de numeración usuales en informática

L2.1 Nociones básicas sobre representación de la información

L2.2 Representación de textos

L2.3 Representación de sonidos

L2.4 Representación de imágenes

L2.5 Representación de vídeo

L2.6 Representación de números enteros

L2.7 Representación de números reales

Para estudiar esta diapositiva

Ahora pasamos de escribir cantidades a decidir qué significa una secuencia de bits. Las convenciones compartidas permiten interpretar, detectar errores y comprimir información.

L2.1

Nociones básicas sobre representación de la información

01 Patrón de bits 02 Redundancia 03 Compresión

Para estudiar esta diapositiva

Esta lección distingue representación, redundancia y compresión. Añadir bits de control y reducir espacio son objetivos distintos; ambos dependen de cómo se organiza la información.

01 · Patrón de bits

Los bits necesitan una interpretación

Un patrón de bits es una secuencia de ceros y unos.



También pueden codificar colores, muestras de sonido o instrucciones.
El código y el contexto determinan su significado.

Para estudiar esta diapositiva

01000001 puede interpretarse como 65 o como la letra A en ASCII. Los bits no contienen por sí solos una etiqueta que indique cuál de esas interpretaciones corresponde.

¿Cuántos bits necesitamos?

Queremos distinguir 5 objetos con códigos de longitud fija.

2 bits → 4 patrones

00	01	10	11
----	----	----	----

No bastan para 5 objetos.

3 bits → 8 patrones

000	001	010	011
100	101	110	111

Bastan; sobran 3 patrones.

Para m objetos: $2^n \geq m \rightarrow n \text{ mínimo} = \lceil \log_2 m \rceil$

Para estudiar esta diapositiva

Con códigos de longitud fija, el número de patrones debe alcanzar el de objetos. Para cinco objetos hacen falta tres bits; algunos patrones pueden quedar sin utilizar.

02 · Redundancia

Añadimos información para controlar errores

Con frecuencia al patrón de bits se le añaden bits adicionales

Datos: n bits

Control: k bits

Total: $n + k$ bits

DETECTAR

Advertir que algo ha cambiado.

CORREGIR

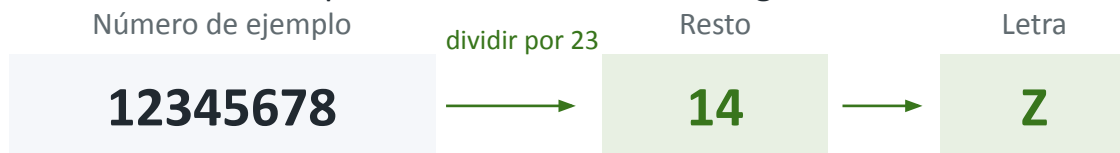
Recuperar los datos originales dentro de los límites del código.

Para estudiar esta diapositiva

Los bits de control no añaden nuevos datos del mensaje: establecen relaciones que ayudan a comprobarlo. Detectar un error no implica saber dónde está ni poder corregirlo.

Ejemplo de redundancia: letra del DNI

La letra se calcula a partir del número; no se elige libremente.



Resto	0	1	2	3	4	5	6	7	8	9	10	11
Letra	T	R	W	A	G	M	Y	F	P	D	X	B
Resto	12	13	14	15	16	17	18	19	20	21	22	
Letra	N	J	Z	S	Q	V	H	L	C	K	E	

Comprobación: recalcular la letra y compararla con la recibida.

Para estudiar esta diapositiva

La letra del ejemplo se obtiene del resto de dividir el número entre 23. Recalcularla permite detectar determinadas incoherencias; no constituye una prueba de identidad ni una corrección automática del número.

Bit de paridad: un bit permite detectar cambios

Paridad par: el total de unos, incluido el bit de control, debe ser par.

	Datos				Paridad	
Enviamos	1	0	1	1	1	4 unos → válido
			↓			
Recibimos	1	0	0	1	1	3 unos → error

También se puede acordar paridad impar.

Un cambio de bit siempre se detecta. Dos cambios pueden pasar inadvertidos.

La paridad por sí sola no indica qué bit hay que corregir.

Para estudiar esta diapositiva

La paridad par exige un número par de unos en datos y control juntos. Detecta cualquier número impar de cambios de bit, pero un número par de cambios puede pasar inadvertido.

Corregir exige más información de control

Un código sencillo repite cada bit tres veces: $0 \rightarrow 000$; $1 \rightarrow 111$.



Se recupera el bit 1 si cambia como máximo un bit del grupo.

En la práctica: checksum y CRC detectan errores; códigos como Hamming permiten corregirlos bajo condiciones concretas, con menos redundancia que repetir cada bit tres veces.

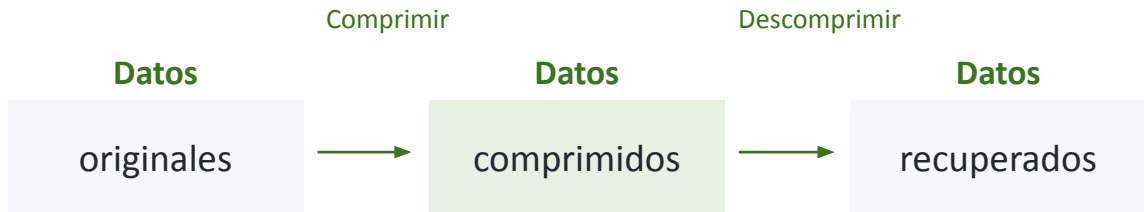
Para estudiar esta diapositiva

La repetición triple permite decidir por mayoría si cambia como máximo un bit de cada grupo. Si cambian dos, la mayoría puede apuntar al valor incorrecto: toda capacidad de corrección tiene límites.

03 · Compresión

Representar los datos con menos espacio

Menos espacio de almacenamiento y menos datos que transmitir.



El algoritmo cambia la representación de los datos.

La recuperación puede ser sin pérdidas o con pérdidas: veremos la diferencia.

Para estudiar esta diapositiva

Comprimir modifica la representación para ahorrar espacio. La distinción esencial es si al descomprimir se recupera exactamente la entrada o solo una aproximación aceptable.

De 14 MB a 5 MB: ¿cuánto hemos reducido?

Un archivo de música ocupa 14 MB; comprimido, ocupa 5 MB.

Antes: 14 MB



Quedan 5 MB

Se ahorran 9 MB

35,7 % del original

64,3 % de ahorro

El original ocupa $14 / 5 = 2,8$ veces el tamaño comprimido.

Factor de compresión: $2,8 : 1$

Para estudiar esta diapositiva

Se conservan 5 de los 14 MB y se eliminan 9. El porcentaje que queda, 35,7 %, y el porcentaje ahorrado, 64,3 %, son cantidades complementarias.

Factor y porcentaje de compresión

C_a : capacidad del archivo antes de comprimir

C_d : capacidad del archivo después de comprimir

Factor de compresión

$$f_c = \frac{C_a}{C_d} = \frac{14 \text{ MB}}{5 \text{ MB}}$$

2,8 : 1

Porcentaje de compresión

$$p_c = \frac{C_d}{C_a} \times 100$$

35,7 % del tamaño inicial

El ahorro es el porcentaje eliminado: $100 - 35,7 = 64,3 \%$.

Para estudiar esta diapositiva

Comprueba siempre qué definición de porcentaje usa el enunciado. Aquí el factor es tamaño inicial dividido por final, mientras que el porcentaje mostrado es la fracción del tamaño inicial que permanece.

Sin pérdidas o con pérdidas: ¿qué recuperamos?

SIN PÉRDIDAS

Se recuperan exactamente los datos originales.

Adecuada cuando cada dato importa: texto, programas, documentos.

Ejemplos: ZIP, PNG.

Algoritmos: RLE, Lempel–Ziv.

CON PÉRDIDAS

Se recupera una aproximación;
se descarta información.

Útil en imagen, audio y vídeo si la pérdida es aceptable.

Ejemplos habituales:

JPEG, MP3 y vídeo MPEG.

Para estudiar esta diapositiva

Sin pérdidas significa recuperar los datos codificados exactamente, no mejorar su calidad. Una imagen que ya haya perdido información no la recupera por guardarse después en un formato sin pérdidas.

Tema 2 · Nuestro recorrido

L2.0 Sistemas de numeración usuales en informática

L2.1 Nociones básicas sobre representación de la información

L2.2 Representación de textos

L2.3 Representación de sonidos

L2.4 Representación de imágenes

L2.5 Representación de vídeo

L2.6 Representación de números enteros

L2.7 Representación de números reales

Para estudiar esta diapositiva

La representación de textos permite aplicar las ideas anteriores a caracteres. Habrá que distinguir el identificador del carácter de los bytes concretos empleados para almacenarlo.

L2.2

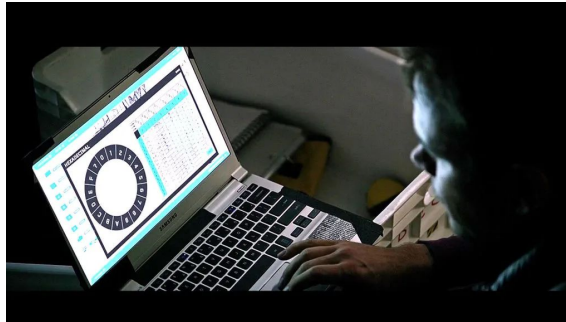
Representación de textos

01 Codificación 02 ASCII 03 Unicode y UTF

Para estudiar esta diapositiva

El recorrido comienza con códigos compartidos, continúa con ASCII y termina con Unicode y sus codificaciones. La diferencia entre carácter y byte será esencial en los ejercicios de tamaño.

¿Cómo enviar texto usando solo números?



«En algún momento, todo se te va a ir al traste... todo se irá al traste y dirás: “ya está, hasta aquí he llegado; así es como termino”. Puedes aceptarlo... o ponerte a trabajar. Eso es todo. Empiezas. Haces las cuentas. Resuelves un problema... y luego el siguiente... y después el siguiente. Y si resuelves suficientes problemas, vuelves a casa.»

— Mark Watney, *The Martian* (2015), Ridley Scott

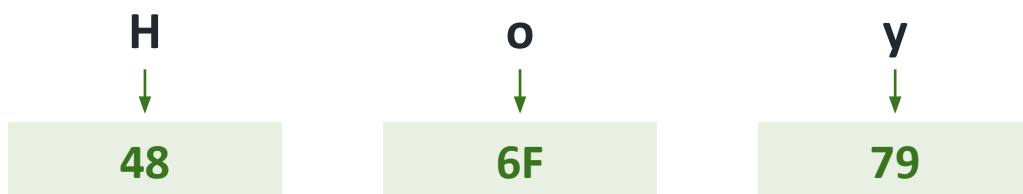
Para estudiar esta diapositiva

La escena sirve de motivación para transmitir texto mediante símbolos limitados. La cuestión informática es acordar una correspondencia estable entre caracteres y números que ambos extremos puedan interpretar.

01 • Codificación

Del texto a sus bytes

Texto «Hoy» guardado en ASCII o UTF-8, sin salto de línea.



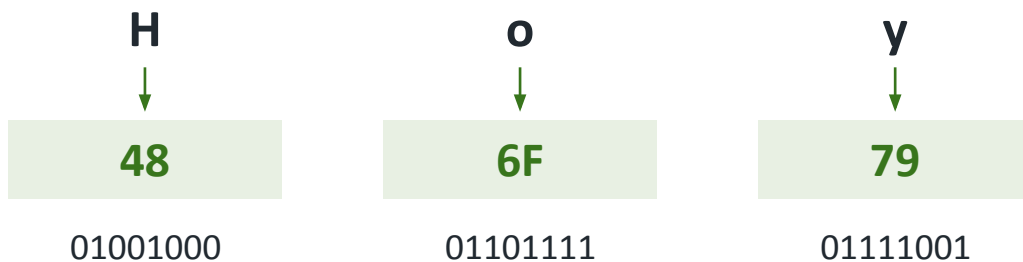
Un editor hexadecimal muestra los bytes del archivo junto a su interpretación como texto. Ejemplos: hexedit, ghex, ...

Para estudiar esta diapositiva

En el ejemplo, los bytes 48, 6F y 79 están escritos en hexadecimal y corresponden a H, o e y. No se ha incluido un salto de línea, que añadiría bytes.

El mismo dato, escrito en hexadecimal o binario

Texto «Hoy» guardado en ASCII o UTF-8, sin salto de línea.



Dos cifras hexadecimales representan un byte: 8 bits.

Para estudiar esta diapositiva

Hexadecimal y binario muestran los mismos bytes con distinta notación. Dos cifras hexadecimales equivalen a ocho bits; no son dos caracteres del texto original.

Para entendernos, debemos compartir el código

Podríamos asignar números a los caracteres de muchas maneras.

Código de Ana

A → 01 B → 10

01



Código de Luis

A → 10 B → 01

Ana envía A; Luis interpreta B.

Una norma compartida fija la correspondencia y permite intercambiar texto.

Para estudiar esta diapositiva

Una misma secuencia puede significar cosas distintas si se utilizan tablas diferentes. La interoperabilidad requiere que emisor y receptor acuerden la codificación.

Un repertorio de 105 caracteres necesita 7 bits

Un punto de código es el número asignado a un carácter dentro de un repertorio.

6 bits

64 patrones

No bastan

7 bits

128 patrones

Bastan; quedan 23 libres

105 caracteres → 105 puntos de código distintos.

$2^6 = 64 < 105 \leq 128 = 2^7$: necesitamos 7 bits.

Para estudiar esta diapositiva

No confundas los 105 caracteres del repertorio con los 128 patrones disponibles. Se elige el menor número de bits que permita asignar un código diferente a cada carácter.

02 · ASCII

Un código de 7 bits y 128 posiciones

Incluye letras latinas sin acentos, cifras, signos y códigos de control.

1	0	0	0	0	0	1
---	---	---	---	---	---	---

A

ASCII: 65 en decimal = 41 en hexadecimal.

En un byte: 01000001

Para estudiar esta diapositiva

ASCII significa American Standard Code for Information Interchange. Usa siete bits para 128 posiciones; al guardarlo en un byte, el bit adicional puede mantenerse a cero, como en 01000001 para A.

Tabla ASCII

ASCII Table — Complete Character Reference

justprotocols.com

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
0	00	NUL	32	20	SPC	64	40	@	96	60	`
1	01	SOH	33	21	!	65	41	A	97	61	a
2	02	STX	34	22	"	66	42	B	98	62	b
3	03	ETX	35	23	#	67	43	C	99	63	c
4	04	EOT	36	24	\$	68	44	D	100	64	d
5	05	ENO	37	25	%	69	45	E	101	65	e
6	06	ACK	38	26	&	70	46	F	102	66	f
7	07	BEL	39	27	'	71	47	G	103	67	g
8	08	BS	40	28	(	72	48	H	104	68	h
9	09	TAB	41	29	)	73	49	I	105	69	i
10	0A	LF	42	2A	*	74	4A	J	106	6A	j
11	0B	VT	43	2B	+	75	4B	K	107	6B	k
12	0C	FF	44	2C	,	76	4C	L	108	6C	l
13	0D	CR	45	2D	-	77	4D	M	109	6D	m
14	0E	SO	46	2E	.	78	4E	N	110	6E	n
15	0F	SI	47	2F	/	79	4F	O	111	6F	o
16	10	DLE	48	30	0	80	50	P	112	70	p
17	11	DC1	49	31	1	81	51	Q	113	71	q
18	12	DC2	50	32	2	82	52	R	114	72	r
19	13	DC3	51	33	3	83	53	S	115	73	s
20	14	DC4	52	34	4	84	54	T	116	74	t
21	15	NAK	53	35	5	85	55	U	117	75	u
22	16	SYN	54	36	6	86	56	V	118	76	v
23	17	ETB	55	37	7	87	57	W	119	77	w
24	18	CAN	56	38	8	88	58	X	120	78	x
25	19	EM	57	39	9	89	59	Y	121	79	y
26	1A	SUB	58	3A	:	90	5A	Z	122	7A	z
27	1B	ESC	59	3B	;	91	5B	[	123	7B	{
28	1C	FS	60	3C	<	92	5C	\	124	7C	
29	1D	GS	61	3D	=	93	5D	]	125	7D	}
30	1E	RS	62	3E	>	94	5E	^	126	7E	~
31	1F	US	63	3F	?	95	5F	_	127	7F	DEL

Control Space Digits Letters Symbols

02 · ASCII

7 / 27

Para estudiar esta diapositiva

La tabla relaciona caracteres y códigos ASCII. Comprueba si sus cabeceras están en decimal o hexadecimal antes de leer una posición; los controles también tienen códigos aunque no dibujen una letra.

Tabla ASCII

ASCII Table — Complete Character Reference

justprotocols.com

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
0	00	NUL	32	20	SPC	64	40	@	96	60	`
1	01	SOH	33	21	!	65	41	A	97	61	a
2	02	STX	34	22	"	66	42	B	98	62	b
3	03	ETX	35	23	#	67	43	C	99	63	c
4	04	END	36	24	\$	68	44	D	100	64	d
5	05	ENO	37	25	%	69	45	E	101	65	e
6	06	ACK	38	26	&	70	46	F	102	66	f
7	07	BEL	39	27	'	71	47	G	103	67	g
8	08	BS	40	28	(	72	48	H	104	68	h
9	09	TAB	41	29	)	73	49	I	105	69	i
10	0A	LF	42	2A	*	74	4A	J	106	6A	j
11	0B	VT	43	2B	+	75	4B	K	107	6B	k
12	0C	FF	44	2C	,	76	4C	L	108	6C	l
13	0D	CR	45	2D	-	77	4D	M	109	6D	m
14	0E	SO	46	2E	.	78	4E	N	110	6E	n
15	0F	SI	47	2F	/	79	4F	O	111	6F	o
16	10	DLE	48	30	0	80	50	P	112	70	p
17	11	DC1	49	31	1	81	51	Q	113	71	q
18	12	DC2	50	32	2	82	52	R	114	72	r
19	13	DC3	51	33	3	83	53	S	115	73	s
20	14	DC4	52	34	4	84	54	T	116	74	t
21	15	NAK	53	35	5	85	55	U	117	75	u
22	16	SYN	54	36	6	86	56	V	118	76	v
23	17	ETB	55	37	7	87	57	W	119	77	w
24	18	CAN	56	38	8	88	58	X	120	78	x
25	19	EM	57	39	9	89	59	Y	121	79	y
26	1A	SUB	58	3A	:	90	5A	Z	122	7A	z
27	1B	ESC	59	3B	;	91	5B	[	123	7B	{
28	1C	FS	60	3C	<	92	5C	\	124	7C	
29	1D	GS	61	3D	=	93	5D	]	125	7D	}
30	1E	RS	62	3E	>	94	5E	^	126	7E	~
31	1F	US	63	3F	?	95	5F	_	127	7F	DEL

Control Space Digits Letters Symbols

02 · ASCII

H

↓

48

o

↓

6F

y

↓

79

Para estudiar esta diapositiva

Localiza H, o e y y recupera los códigos 48, 6F y 79 en hexadecimal. Reconstruir «Hoy» ayuda a practicar la lectura de la tabla en ambos sentidos.

ASCII contiene caracteres visibles y controles

00–1F

Control

LF: salto de línea

20

Espacio

separa palabras

30–39

Cifras

0 1 2 ... 9

41–5A

Mayúsculas

A B C ... Z

61–7A

Minúsculas

a b c ... z

7F

Control

DEL

Los códigos están en hexadecimal. El carácter «0» tiene código 30; no es el número 0.

Para estudiar esta diapositiva

El carácter «0» tiene código hexadecimal 30; no es un byte de valor cero. Los códigos de control tienen funciones de organización o comunicación, como el salto de línea.

Leemos «j» en la tabla ASCII: fila 6, columna A

Fragmento de la tabla: primer dígito hexadecimal en la fila; segundo en la columna.

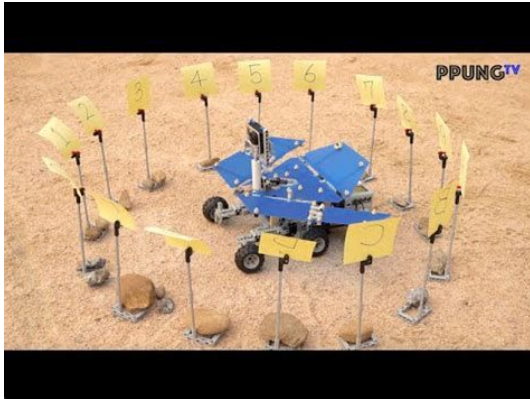
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
→ 6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o

j → 6A₁₆ → 106₁₀ → 01101010₂

Para estudiar esta diapositiva

La fila aporta la primera cifra hexadecimal y la columna, la segunda. $6A_{16} = 6 \times 16 + 10 = 106$, que se escribe 01101010 en un byte.

Con 16 símbolos {0-F} podemos enviar cualquier byte



Un dígito hexadecimal
tiene 16 valores: 0–F.

4

8

 $48_{16} \rightarrow H$

En la escena, dos selecciones hexadecimales permiten comunicar un carácter ASCII.

Para estudiar esta diapositiva

Dos selecciones entre 16 símbolos permiten comunicar $16 \times 16 = 256$ combinaciones. Para representar un carácter ASCII mediante dos cifras hexadecimales se está describiendo su byte, no enviando dos letras del mensaje.

Ampliaciones de ASCII: de 7 a 8 bits (1/2)

128 → 256 valores: conservan ASCII y añaden símbolos para otras lenguas. La tabla elegida determina su significado.

Denominación	Estándar	Área geográfica / escritura
Latin-1	ISO 8859-1	Europa occidental
Latin-2	ISO 8859-2	Europa central y oriental
Latin-3	ISO 8859-3	Sur de Europa y esperanto
Latin-4	ISO 8859-4	Norte de Europa
Latino / cirílico	ISO 8859-5	Lenguas con alfabeto cirílico
Latino / árabe	ISO 8859-6	Árabe
Latino / griego	ISO 8859-7	Griego moderno
Latino / hebreo	ISO 8859-8	Hebreo

Para estudiar esta diapositiva

Las extensiones de ocho bits ampliaron el repertorio, pero no todas asignaron los mismos caracteres a los códigos superiores. Para interpretar un archivo antiguo hay que conocer la tabla concreta utilizada.

Ampliaciones de ASCII: de 7 a 8 bits (2/2)

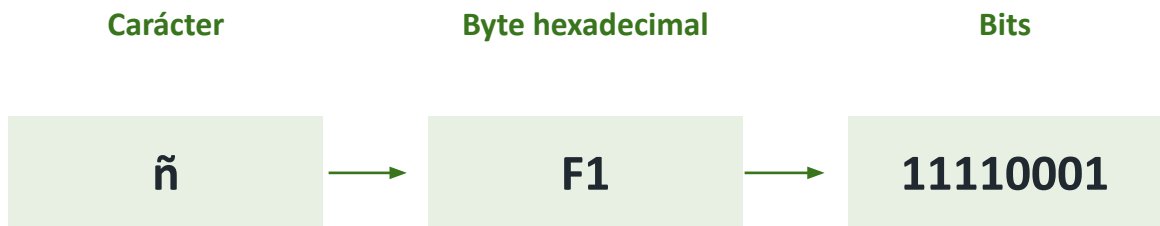
128 → 256 valores: conservan ASCII y añaden símbolos para otras lenguas. La tabla elegida determina su significado.

Denominación	Estándar	Área geográfica / escritura
Latin-5	ISO 8859-9	Turco
Latin-6	ISO 8859-10	Lenguas nórdicas
Latino / tailandés	ISO 8859-11	Tailandés
Latin-7	ISO 8859-13	Lenguas bálticas
Latin-8	ISO 8859-14	Lenguas celtas
Latin-9	ISO 8859-15	Latin-1 revisado; incorpora €
Latin-10	ISO 8859-16	Sudeste de Europa

Para estudiar esta diapositiva

Las distintas familias ISO 8859 responden a repertorios diferentes. Compartir un byte de almacenamiento no garantiza compartir el significado de todos sus valores.

Latin-1 permite representar «ñ» con un byte



Esta correspondencia pertenece a ISO-8859-1 (Latin-1).

Para estudiar esta diapositiva

F1 representa ñ en Latin-1. Esa equivalencia no autoriza a interpretar F1 como un byte UTF-8 independiente: la codificación forma parte de la información necesaria.

Un repertorio de 256 posiciones se queda pequeño

Queremos mezclar alfabetos y símbolos en un mismo texto.



**¿Cómo mezclamos letras, ideogramas chinos
y escritura árabe sin cambiar de tabla?**

Necesitamos un repertorio común para todas estas escrituras.

Para estudiar esta diapositiva

Un repertorio de 256 posiciones no basta para combinar todas las escrituras.
Cambiar constantemente de tabla dificulta el intercambio de documentos multilingües.

03 · Unicode y sus codificaciones

Un número para identificar cada carácter

Letra latina

A

U+0041

Aum · devanagari

ॐ

U+0950

Yuè · chino

月

U+6708

Unicode asigna un punto de código a cada carácter.

U+ introduce su número escrito en hexadecimal.

[Ejemplos: Wikipedia · Unicode](#)

Para estudiar esta diapositiva

U+ identifica un punto de código escrito en hexadecimal. Es una etiqueta del repertorio Unicode; todavía no especifica cuántos bytes ocupará el carácter en un archivo.

Unicode: universalidad, unicidad y uniformidad

Universalidad

Un repertorio común para los sistemas de escritura y símbolos de distintas lenguas.

Unicidad

Cada carácter codificado tiene su identificador:
A es U+0041, con independencia del sistema o programa.

Uniformidad · matiz histórico

La idea inicial era usar 16 bits por carácter. Hoy Unicode se representa mediante UTF-8, UTF-16 o UTF-32.

Un carácter visible puede combinar varios puntos de código. Unicode no implica «siempre 2 bytes».

[Fuente: Unicode Consortium · Introducción y cap. 2](#)

16 / 29

Para estudiar esta diapositiva

Unicode no equivale a «dos bytes por carácter». También conviene distinguir punto de código y carácter visible: algunas formas visibles se construyen combinando varios puntos de código.

Unicode: zonas del Plano Básico Multilingüe (BMP)

BMP: U+0000–U+FFFF · 65.536 posiciones de código

Zona orientativa	Códigos (hex.)	Contenido / ejemplos	Posiciones
Latín inicial	0000–00FF	ASCII y suplemento Latin-1 · A, ñ	256
Escrituras	0100–1FFF	Alfabetos y signos · griego, árabe...	7.936
Símbolos	2000–33FF	Puntuación, matemáticas, kana...	5.120
Ideogramas	3400–9FFF	CJK · chino, japonés y coreano	27.648
Otras escrituras	A000–D7FF	Yi, hangul y otras escrituras	14.336
Sustitutos	D800–DFFF	Reservados para UTF-16; no caracteres	2.048
Uso privado	E000–F8FF	Significado acordado por los usuarios	6.400
Compatibilidad	F900–FFFF	Compatibilidad y códigos especiales	1.792

El número indica posiciones del intervalo, no caracteres ya asignados.

Las zonas son orientativas; Unicode incluye otros 16 planos fuera del BMP.

[Fuente: Unicode, cap. 2.8–2.9](#)

17 / 29

Para estudiar esta diapositiva

El BMP es un plano del espacio Unicode, no el espacio completo. Los tamaños de los intervalos cuentan posiciones posibles; no significan que todas estén asignadas a caracteres.

Identificar un carácter no fija cómo guardarlo

El carácter ñ siempre tiene el punto de código U+00F1.

Punto de código

ñ → U+00F1

Codificación en memoria

UTF-8 o UTF-16

Para escribirlo en un archivo debemos elegir cómo convertir ese número en una secuencia de bytes.

Mismo carácter y mismo número; los bytes dependen de la codificación.

Para estudiar esta diapositiva

El identificador U+00F1 se conserva al elegir UTF-8 o UTF-16, pero cambia su representación en bytes. Este es un ejemplo de separar significado y forma de almacenamiento.

UTF-8: convertir puntos de código en bytes

El rango del punto de código determina cuántos bytes necesita.

Punto de código	Bytes	Patrón de bits
U+0000–U+007F	1	0xxxxxxx
U+0080–U+07FF	2	110xxxxx 10xxxxxx
U+0800–U+FFFF	3	1110xxxx 10xxxxxx 10xxxxxx
U+10000–U+10FFFF	4	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx

Las x contienen los bits del número. ASCII conserva sus bytes.

Para estudiar esta diapositiva

Los prefijos indican la estructura de UTF-8 y las x contienen el valor codificado. No todos los números del intervalo de tres bytes son valores escalares válidos: los sustitutos U+D800–U+DFFF están excluidos. Referencia: https://www.unicode.org/faq/utf_bom.html .

Codificar la ñ en UTF-8 · 1. Preparar los bits

Unicode: ñ → U+00F1. Su número es $F1_{16} = 241_{10}$.

U+00F1 está entre U+0080 y U+07FF: necesita 2 bytes.

El patrón reserva 5 + 6 = 11 bits para el número:



Hay 11 huecos x: completamos con 3 ceros a la izquierda.

El valor sigue siendo 241; solo preparamos los bits para repartirlos.

Para estudiar esta diapositiva

241 necesita ocho bits de valor, pero el patrón UTF-8 de dos bytes ofrece once huecos. Se añaden ceros a la izquierda para completar esos huecos sin modificar el número.

Codificar la ñ en UTF-8 · 2. Formar los bytes

Repartimos los 11 bits en dos grupos: 00011 y 110001.

PRIMER BYTE

00011

↓ añadimos 110

11000011

C3

SEGUNDO BYTE

110001

↓ añadimos 10

10110001

B1

Azul: prefijos UTF-8. Verde: bits del carácter. ñ → C3 B1 (2 bytes)

Para estudiar esta diapositiva

Los prefijos 110 y 10 son estructurales; no pertenecen al valor 241. Al retirarlos y reunir 00011 y 110001 se recupera el punto de código original.

UTF-16: unidades de 16 bits

Cada unidad contiene 16 bits, es decir, 2 bytes.

Muchos caracteres usan una unidad

ñ → U+00F1 → 00000000 11110001

ñ ocupa 2 bytes en UTF-16. También Å, 𐍚 y 月.

Otros caracteres, como muchos emojis, necesitan dos unidades: 32 bits = 4 bytes.

Para estudiar esta diapositiva

UTF-16 utiliza unidades de 16 bits. Los caracteres fuera del BMP requieren un par de unidades; por ello no debe aplicarse siempre la regla de dos bytes a cualquier texto Unicode.

1. Factor de compresión al pasar a UTF-8

Un archivo de texto contiene estos caracteres:

18.325

U+0000–U+007F

127

U+0080–U+07FF

14

U+0800–U+FFFF

¿Qué factor de compresión obtenemos al recodificar el archivo de UTF-16 a UTF-8?

Por simplicidad, suponemos 2 bytes por carácter en UTF-16.

Para estudiar esta diapositiva

Calcula primero el número total de caracteres y después los bytes de cada codificación. La hipótesis de dos bytes por carácter en UTF-16 es parte explícita de este ejercicio.

i) Aplicamos la tabla de UTF-8

Punto de código	Bytes	Patrón de bits
U+0000–U+007F	1	0xxxxxxx
U+0080–U+07FF	2	110xxxxx 10xxxxxx
U+0800–U+FFFF	3	1110xxxx 10xxxxxx 10xxxxxx
U+10000–U+10FFFF	4	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx

$18.325 \times 1 = 18.325 \text{ B}$ $127 \times 2 = 254 \text{ B}$ $14 \times 3 = 42 \text{ B}$

UTF-8: $18.325 + 254 + 42 = 18.621 \text{ B}$

Para estudiar esta diapositiva

Cada grupo se multiplica por su coste de uno, dos o tres bytes. Suma tamaños en bytes, no cantidades de caracteres con cantidades de bytes.

ii) Calculamos el factor de compresión

$18.325 + 127 + 14 = 18.466$ caracteres.

UTF-16

$$18.466 \times 2 = 36.932 \text{ B}$$

UTF-8

$$18.621 \text{ B}$$

$$f_c = 36.932 / 18.621 \approx 1,98 : 1$$

El archivo en UTF-8 ocupa aproximadamente la mitad.

Para estudiar esta diapositiva

El factor $36.932/18.621 \approx 1,98$ compara tamaños del mismo texto. Aquí se habla de recodificación, no de aplicar un compresor general al archivo.

2. Del tamaño del archivo al número de caracteres

Un archivo UTF-8 ocupa 139 KiB y contiene:

1.024 caracteres de 3 bytes

4.096 caracteres de 2 bytes

El resto ocupa 1 byte por carácter.

¿Cuánto ocupará en UTF-16?

Supón 2 bytes por carácter.

a) 266 KiB b) 264 KiB c) 278 KiB d) 261 KiB

26 / 29

Para estudiar esta diapositiva

Convierte 139 KiB a bytes y descuenta los grupos cuyo tamaño conoces. Solo entonces podrás deducir cuántos caracteres de un byte quedan.

i) Calculamos los grupos conocidos

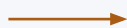
Tamaño total en UTF-8: 139 KiB. 1 KiB = 1.024 B.

$$1.024 \times 3 \text{ B}$$



3 KiB

$$4.096 \times 2 \text{ B}$$



8 KiB

Los dos grupos ocupan $3 + 8 = 11$ KiB.

Para estudiar esta diapositiva

Los dos grupos conocidos ocupan 3 y 8 KiB. La unidad KiB equivale a 1024 bytes; mantener esa convención evita mezclarla con kB decimal.

ii) El espacio restante revela cuántos faltan

Los grupos de 3 y 2 bytes ocupan 11 KiB de los 139 KiB.

$$139 \text{ KiB} - 11 \text{ KiB} = 128 \text{ KiB}$$



$$128 \times 1.024 = 131.072 \text{ bytes}$$

→ 131.072 caracteres de un byte

Para estudiar esta diapositiva

Los 128 KiB restantes son 131.072 bytes. Como cada carácter restante cuesta un byte, coinciden numéricamente el número de bytes y el de caracteres de ese grupo.

iii) En UTF-16 suponemos 2 bytes por carácter

Contamos los caracteres y multiplicamos por 2 bytes.

$$1.024 + 4.096 + 131.072 = 136.192 \text{ caracteres}$$



$$136.192 \times 2 \text{ B} = 272.384 \text{ B}$$

$$272.384 / 1.024 = 266 \text{ KiB} \rightarrow \text{opción a)}$$

Para estudiar esta diapositiva

Suma todos los caracteres, incluidos los grupos ya descontados, antes de multiplicar por dos. El resultado de 266 KiB corresponde a las hipótesis del enunciado, sin añadir cabeceras.

Tema 2 · Nuestro recorrido

L2.0 Sistemas de numeración usuales en informática

L2.1 Nociones básicas sobre representación de la información

L2.2 Representación de textos

L2.3 Representación de sonidos

L2.4 Representación de imágenes

L2.5 Representación de vídeo

L2.6 Representación de números enteros

L2.7 Representación de números reales

Para estudiar esta diapositiva

El siguiente bloque introduce señales que varían con el tiempo. Para almacenarlas habrá que decidir cuándo medir y con cuántos niveles representar cada medición.

L2.3

Representación de sonidos

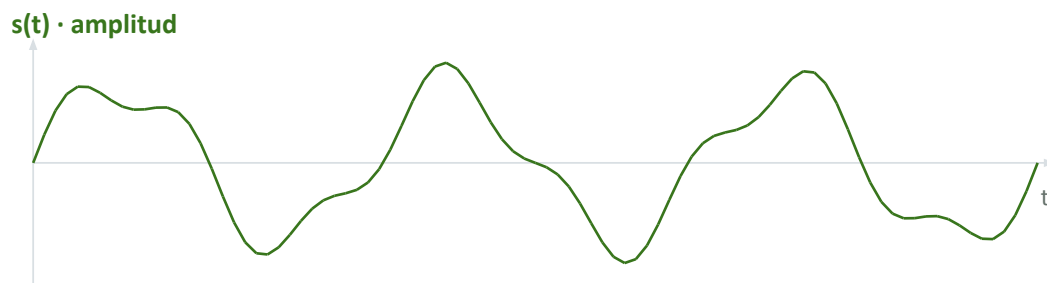
01 Digitalizar 02 Tamaño y formatos 03 Transmitir y reproducir

Para estudiar esta diapositiva

En sonido se separan muestreo temporal, cuantificación de amplitud y codificación.
Después se relacionan esos parámetros con la tasa de bits y el tamaño del archivo.

01 · El sonido es una señal continua $s(t)$

En cada canal, la amplitud cambia continuamente con el tiempo.



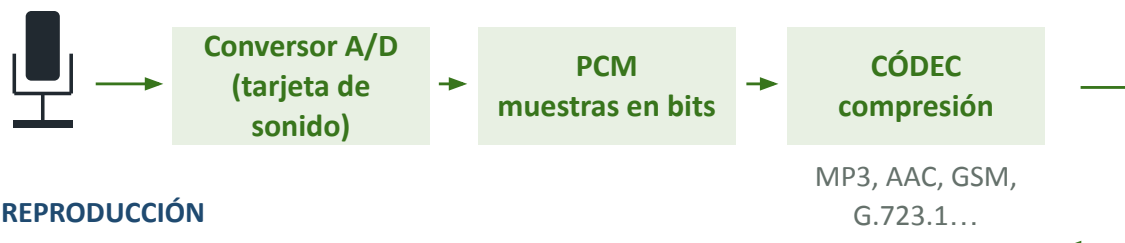
Digitalizar exige discretizar el tiempo y la amplitud: muestreo y cuantificación.

Para estudiar esta diapositiva

Una señal analógica se describe como una función continua del tiempo. Digitalizar implica elegir instantes de medida y un conjunto finito de valores para expresar las amplitudes.

Grabación y reproducción: el proceso completo

GRABACIÓN



REPRODUCCIÓN

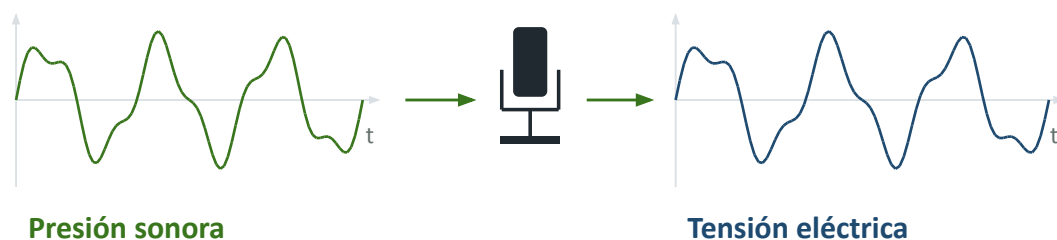


Para estudiar esta diapositiva

Sigue por separado la ruta de grabación y la de reproducción. El conversor trata la relación entre señales físicas y muestras; el códec trata la codificación de los datos.

Captación: de la presión a una señal eléctrica

Un micrófono es un transductor: transforma la variación de presión del aire en una señal eléctrica analógica.



La señal analógica puede tomar cualquier valor dentro de un intervalo continuo.

Para estudiar esta diapositiva

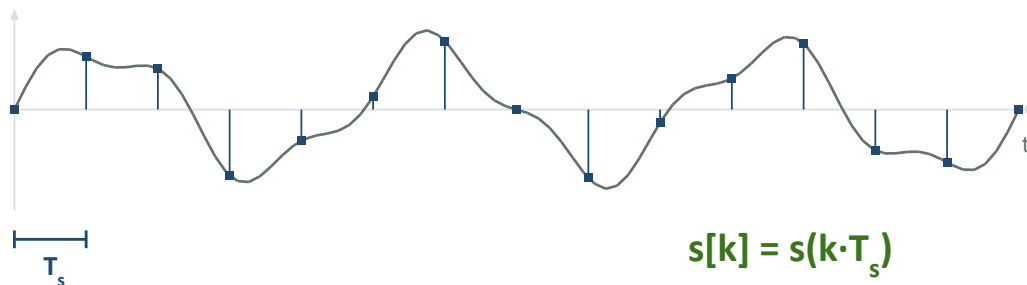
Un transductor cambia una magnitud física en otra. El micrófono produce una señal eléctrica relacionada con la presión sonora; por sí solo no convierte esa señal en números binarios.

Muestreo: guardar valores en instantes regulares

Hay infinitos instantes: no podemos guardarlos todos en formato digital.

Tomamos una muestra cada período T_s .

$$f_s = \frac{1}{T_s}$$



Ejemplo: $f_s = 44.100$ muestras/s $\rightarrow T_s \approx 22,7 \mu\text{s}$.

Para estudiar esta diapositiva

Frecuencia y período son inversos: aumentar las muestras por segundo reduce la separación temporal. Comprueba las unidades: 1/Hz se expresa en segundos.

Muestreo: reconstrucción y compromiso de tamaño

La frecuencia de muestreo debe permitir reconstruir la señal.
Si aumentamos f_s , guardamos más muestras por segundo.

Teorema de Nyquist–Shannon

Para una señal limitada a B Hz, el límite teórico es $2B$.
En la práctica usamos $f_s > 2B$ y filtramos antes de muestrear.

Audio hasta 20 kHz → más de 40.000 muestras/s

B es la frecuencia máxima presente (ancho de banda de una señal de banda base).

[Explorar el muestreo ↗](#)

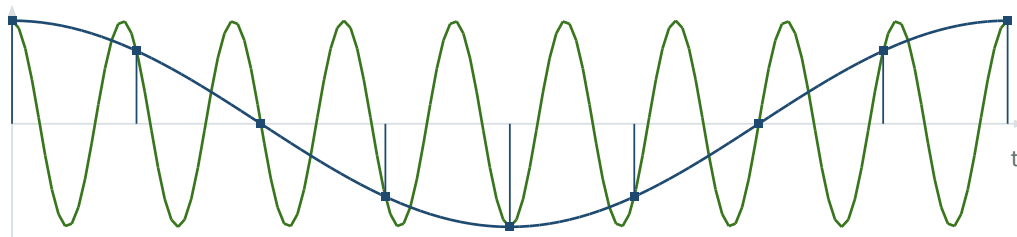
5 / 24

Para estudiar esta diapositiva

La condición de muestreo depende de la frecuencia más alta presente en la señal.
Filtrar antes de muestrear evita que componentes demasiado rápidas se confundan con otras más lentas.

Si muestreamos poco, dos señales pueden confundirse

Mismas muestras; dos ondas distintas. Esto es aliasing.



Verde: 9 Hz Azul: 1 Hz Muestreo: 8 muestras/s

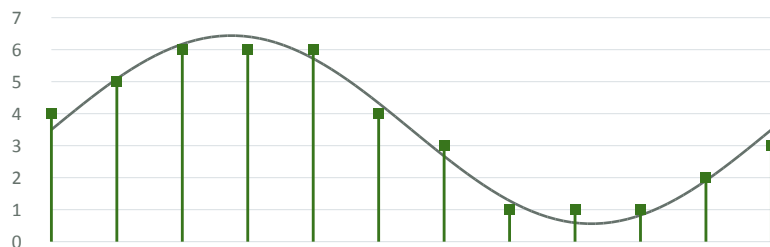
Las muestras no permiten distinguirlas: la frecuencia aparente puede ser incorrecta.

Para estudiar esta diapositiva

Las muestras aisladas pueden ser compatibles con distintas ondas. El aliasing no se resuelve aumentando solo los bits de amplitud: el problema es la separación temporal entre muestras.

Cuantificación: aproximar la amplitud de cada muestra

La amplitud continua se asigna a uno de un conjunto finito de niveles.



$L = 2^n$
3 bits → 8
niveles

Ejemplo con 3 bits: 8 niveles. El redondeo introduce error de cuantificación.

Para estudiar esta diapositiva

Con n bits hay 2^n códigos disponibles para niveles de amplitud. Cuantificar aproxima la muestra a uno de esos niveles e introduce un error que debe distinguirse del error de muestreo.

Más bits por muestra: más niveles, menor error

La resolución n_b controla la precisión de la amplitud



Conclusión: Más bits reducen el error de cuantificación (aumentan la fidelidad), pero incrementan la tasa de transferencia (bitrate) y el tamaño del archivo.

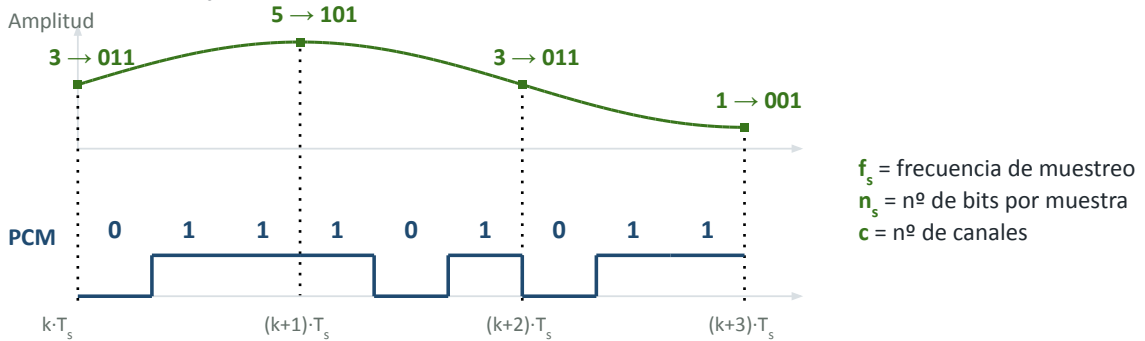
Para estudiar esta diapositiva

Más bits permiten niveles de amplitud más finos para un mismo intervalo. Las etiquetas de calidad son orientativas: la fidelidad también depende del muestreo, del ruido y del resto del sistema.

PCM: transmitir los bits entre dos muestras

PCM · Pulse Code Modulation (modulación por impulsos codificados)

Cada intervalo T_s permite transmitir la palabra de la muestra tomada al inicio.



Caudal o tasa de bits (bit rate):

$$R_{bps} = f_s \cdot n_s \cdot c \text{ (bit/s)}$$

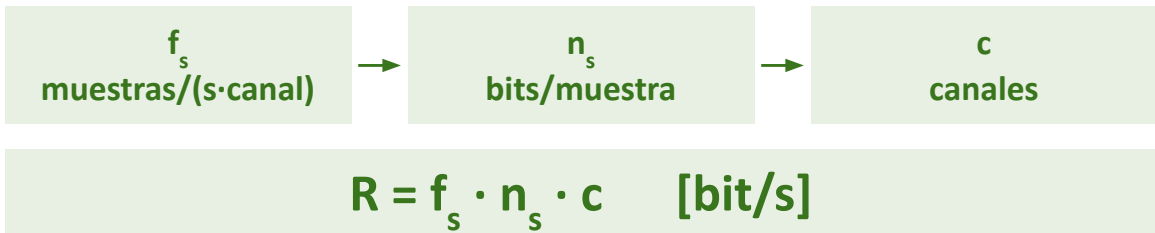
9 / 24

Para estudiar esta diapositiva

En PCM se asigna una palabra binaria a cada muestra. La tasa sin comprimir se obtiene multiplicando muestras por segundo, bits por muestra y número de canales.

Caudal o tasa de bits: cuántos bits por segundo

Cada segundo guardamos f_s muestras en cada uno de c canales.



CD estéreo: $44.100 \times 16 \times 2 = 1.411.200$ bit/s

PCM sin comprimir. La tasa de un códec comprimido depende de su codificación.

Para estudiar esta diapositiva

El factor de canales multiplica la cantidad de datos: estéreo tiene dos canales. La fórmula se refiere a PCM sin comprimir y no permite deducir directamente la tasa de un MP3.

Códecs: qué se codifica en cada método

PCM · Pulse Code Modulation

Codifica la amplitud de cada muestra como una palabra binaria.

DPCM · Differential Pulse Code Modulation

Codifica la diferencia respecto a la muestra anterior

ADPCM · Adaptive Differential Pulse Code Modulation

Se predice el nuevo valor, y se almacena el error entre el valor predicho y el real

μ-law y A-law · PCM no uniforme

Dedican mayor precisión a amplitudes pequeñas (compansión).

G.711: 8.000 muestras/s × 8 bits = 64 kbit/s por canal.

MP2, MP3 y AAC utilizan compresión perceptual: se explica a continuación.

Para estudiar esta diapositiva

Estas familias difieren en qué representan: amplitudes, diferencias o errores de predicción. La adaptación de ADPCM modifica parámetros de la codificación; no consiste únicamente en restar dos muestras consecutivas.

Códecs y compresión: conservar o descartar información

Sin pérdida · FLAC

Aprovecha redundancias.
Recupera exactamente las
muestras PCM de entrada.

Con pérdida · MP3, AAC

Reduce también información
poco perceptible mediante
modelos de audición.

Otras familias del original: MP2, Vorbis, WMA y códecs de voz como GSM o G.723.1. El códec se elige según el uso.

Para estudiar esta diapositiva

FLAC permite recuperar las muestras PCM de entrada, mientras que MP3 y AAC suelen descartar información perceptualmente menos relevante. La recuperación exacta se refiere a los datos de entrada, no a una señal analógica ideal.

Ejemplos: parámetros, tasa y capacidad de audio

Ejemplo	f_s (kHz)	bits	canales	R (kbit/s)	1 min (MB)
Telefonía	8	8	1	64	0,480
Radio AM*	11	8	1	88	0,660
Radio FM*	22,05	8	2	352,8	2,646
CD	44,1	16	2	1.411,2	10,584
DVD / TV*	48	16	2	1.536	11,520

$$R_{\text{bps}} = f_s \cdot n_s \cdot c$$

* Configuraciones ilustrativas, no límites ni normas universales. MB decimales.

13 / 24

Para estudiar esta diapositiva

Puedes verificar cada fila multiplicando frecuencia, bits y canales. Las configuraciones marcadas son ejemplos ilustrativos; no deben interpretarse como requisitos universales de todos los equipos de esa categoría.

Tamaños de archivo de audio: cabecera y muestras codificadas

Cabecera
formato y parámetros

Datos de audio
muestras sucesivas

$$C(t) = \frac{f_s \cdot n_s \cdot c \cdot t}{8}$$

C(t): capacidad de almacenamiento necesaria para los datos en función del tiempo t, expresada en bytes (sin cabecera).

**1 minuto de CD: $44.100 \times 16 \times 2 \times 60 \div 8$
= 10.584.000 B = 10,584 MB $\approx$ 10,094 MiB**

Para estudiar esta diapositiva

Multiplicar la tasa de bits por la duración produce bits; dividir entre ocho produce bytes. El cálculo excluye la cabecera y distingue MB decimales de MiB binarios.

Formatos de sonido: organización y codificación

Formato	Extensión	Contenido / observación
WAV	.wav	Contenedor; a menudo PCM.
AIFF	.aif / .aiff	A menudo PCM; AIFF-C admite compresión.
AU	.au	PCM, μ -law u otras codificaciones.
Audio-CD	CD-DA	PCM estéreo, 44,1 kHz y 16 bits.
FLAC	.flac	Compresión sin pérdida.

El contenedor organiza los datos; el códec determina cómo se codifica el audio.

Para estudiar esta diapositiva

Contenedor y codificación no son sinónimos. Una extensión orienta sobre la organización del archivo, pero no siempre determina un único método de codificación de sus muestras.

Formatos comprimidos: el tamaño depende de la tasa

Familia	Extensión habitual	Compresión
MP3	.mp3	Con pérdida · MPEG Audio Layer III
AAC	.aac / .m4a	Con pérdida
Vorbis	.ogg	Con pérdida; suele usar Ogg
WMA / WMA Lossless	.wma	Con pérdida / sin pérdida

A 128 kbit/s: 1 minuto ocupa $\approx$ 0,960 MB de audio.

No existe un porcentaje fijo: depende del contenido, el códec y sus ajustes.

Para estudiar esta diapositiva

Con tasa constante, tamaño y duración son proporcionales. El cálculo de 0,960 MB por minuto a 128 kbit/s se refiere a los datos de audio, sin contar toda la sobrecarga del archivo.

03 · Transmisión: descargar o escuchar en streaming

Descarga

Recibimos el archivo.
Podemos reproducirlo
después de descargarlo.

Streaming

Empezamos a reproducir
mientras siguen llegando
los datos.

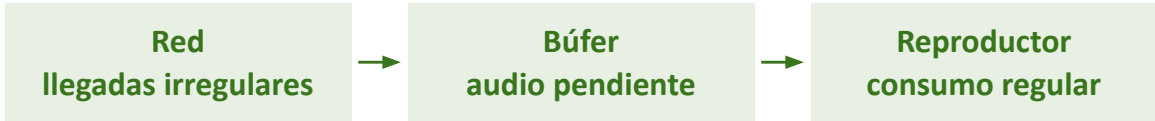
Tasa útil media de recepción > tasa del audio

Se necesita margen para retrasos y sobrecarga; el doble no es una condición universal.

Para estudiar esta diapositiva

En streaming se solapan recepción y reproducción. La tasa media útil debe sostener el consumo durante la reproducción, pero también importan las variaciones temporales y la reserva del búfer.

El búfer permite recibir en ráfagas y reproducir a ritmo fijo



Audio acumulado



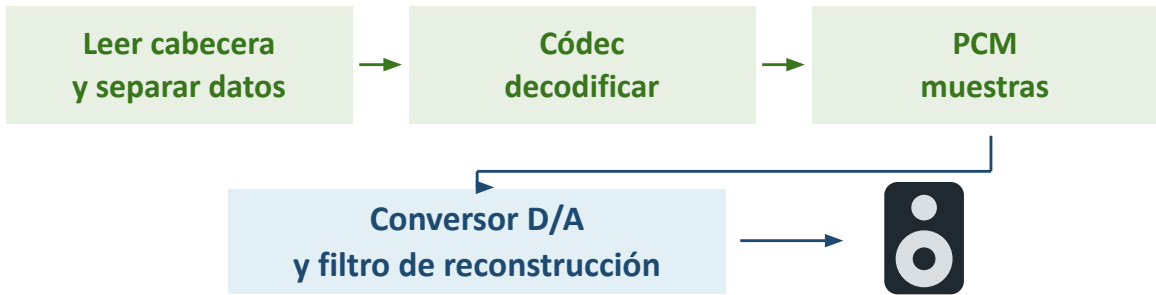
Si el búfer llega a cero, la reproducción se interrumpe. Más reserva implica más espera inicial.

Para estudiar esta diapositiva

El búfer desacopla llegadas irregulares y consumo regular. Una reserva mayor tolera interrupciones más largas, a cambio de aumentar la espera inicial o el retraso de reproducción.

Reproducción: reconstruir la señal al ritmo adecuado

Los datos deben llegar a tiempo y las muestras reproducirse con la frecuencia f_s indicada en la cabecera.



Conversor D/A → señal eléctrica continua → altavoz → variaciones de presión sonora.

Para estudiar esta diapositiva

La frecuencia de reproducción debe corresponder a la de las muestras. La cadena termina reconstruyendo una señal eléctrica que el altavoz transforma en variaciones de presión.

1. Deducimos la frecuencia de muestreo

Un audio estéreo dura 20 s, usa 16 bits por muestra y ocupa 625 KiB de datos PCM sin comprimir.

¿A qué frecuencia se ha muestreado?

a) 8 kHz b) 22,05 kHz c) 44,1 kHz d) 1 kHz

Para estudiar esta diapositiva

Identifica los datos: dos canales, 16 bits por muestra y 20 segundos. Antes de despejar la frecuencia, expresa los 625 KiB en bytes.

i) Partimos de la fórmula del tamaño y despejamos

$$C(t) = \frac{f_s \cdot n_s \cdot c \cdot t}{8}$$

$C(t)$: bytes de datos · f_s : muestras/s · n_s : bits/muestra
 c : número de canales · t : duración en segundos

$$f_s = \frac{8 \cdot C(t)}{n_s \cdot c \cdot t}$$

Datos: $C(t) = 625 \text{ KiB}$ · $n_s = 16 \text{ bits}$ · $c = 2$ · $t = 20 \text{ s}$

Para estudiar esta diapositiva

Despejar antes de sustituir ayuda a controlar las unidades. En $f_s = 8C/(n_s \cdot c \cdot t)$, el ocho convierte bytes a bits y el número de canales evita contar el estéreo como mono.

ii) Sustituimos los datos y calculamos la frecuencia

Convertimos la capacidad a bytes:

$$C(t) = 625 \times 1.024 = 640.000 \text{ B}$$

$$f_s = \frac{8 \times 640.000}{16 \times 2 \times 20} = 8.000 \text{ Hz}$$

Respuesta a) 8 kHz · 8.000 muestras por segundo en cada canal.

Para estudiar esta diapositiva

625×1024=640.000 bytes; la sustitución da 8.000 muestras por segundo en cada canal. Una comprobación consiste en reconstruir el tamaño con la frecuencia obtenida.

2. ¿Qué condición permite reconstruir la señal?

Según Nyquist–Shannon, ¿cuál es el límite mínimo de muestreo para una señal cuya frecuencia máxima es $f_{\max}$?

- a) El doble de la frecuencia máxima de la señal.
- b) La mitad de la frecuencia máxima.
- c) La misma frecuencia que la máxima.
- d) Siempre una frecuencia superior a 22 kHz.

Para estudiar esta diapositiva

La pregunta se refiere a la mayor frecuencia presente, no a la amplitud más alta. No hay una frecuencia mínima universal válida para cualquier señal.

El límite de Nyquist-Shannon es el doble de la frecuencia máxima

Respuesta a): el límite de Nyquist-Shannon es $2 f_{\max}$.

Lo relevante es la frecuencia más alta presente en la señal, no la de mayor amplitud.

Para evitar casos límite y permitir el filtrado, en la práctica $f_s > 2 f_{\max}$.

Para estudiar esta diapositiva

$2f_{\max}$ expresa el límite teórico del planteamiento. En una implementación práctica se necesita margen y filtrado; no conviene interpretar la igualdad como garantía para cualquier señal y fase.

Tema 2 · Nuestro recorrido

L2.0 Sistemas de numeración usuales en informática

L2.1 Nociones básicas sobre representación de la información

L2.2 Representación de textos

L2.3 Representación de sonidos

L2.4 Representación de imágenes

L2.5 Representación de vídeo

L2.6 Representación de números enteros

L2.7 Representación de números reales

Para estudiar esta diapositiva

Las imágenes añaden una dimensión espacial al problema de representar información. Veremos cómo percepción, resolución y profundidad de color condicionan los datos que se guardan.

L2.4

Representación de imágenes

01 Percepción 02 Representaciones
03 Formatos y tamaño 04 Compresión

Para estudiar esta diapositiva

Distingue cómo se percibe una imagen, cómo se describe y cómo se comprime. El tamaño de los datos sin comprimir puede calcularse antes de elegir un formato de archivo.

01 · Física y fisiología de la percepción visual

Cuatro propiedades ayudan a entender cómo representamos imágenes.

P1 · Integración espacial

Cómo percibimos detalles próximos.

P2 · Integración temporal

Cómo percibimos secuencias de imágenes.

P3 · Teoría del color

Cómo describimos y reproducimos colores.

P4 · Sensibilidad en la percepción de colores

Qué información visual distinguimos mejor.

Para estudiar esta diapositiva

Estas propiedades motivan decisiones de representación, pero no son equivalencias exactas entre percepción humana y almacenamiento. Sirven para entender por qué ciertas simplificaciones resultan aceptables.

Integración espacial

El ojo tiene una resolución espacial limitada: no siempre distingue puntos o zonas muy próximos.

Visión de conjunto · Percepción continua



Detalle cercano · Trazo individual



Las pinceladas que distinguimos de cerca se integran al observar el conjunto.

2 / 43

Para estudiar esta diapositiva

Al alejarnos dejamos de resolver detalles individuales y percibimos un conjunto. La comparación ilustra la resolución espacial limitada; no implica que los detalles hayan desaparecido físicamente de la imagen.

«**Las amapolas**» (título original en francés: *Coquelicots* o *Coquelicots, la promenade*), pintada por **Claude Monet** en **1873**

Integración temporal

Una sucesión de imágenes fijas puede percibirse como movimiento.

Persistencia retiniana

La respuesta visual persiste brevemente tras la desaparición del estímulo.

Procesamiento cerebral

El movimiento aparente también depende de la capacidad del cerebro para sintetizar la secuencia.

Ejemplo · Secuencia de movimiento



Cine y vídeo: 24, 25, 30 fps... La fluidez y el parpadeo dependen de las condiciones.

3 / 43

Para estudiar esta diapositiva

Una secuencia de imágenes puede producir movimiento aparente; no basta atribuir todo el fenómeno a la persistencia de la retina. Como complemento al comentario histórico previo, las fotografías de Muybridge permiten analizar fases de un movimiento que a simple vista cuesta separar. La Library of Congress conserva la serie de Sallie Gardner, realizada en Palo Alto el 19 de junio de 1878, vinculada a Leland Stanford. Fuente: <https://www.loc.gov/item/97502309/> .

Teoría del color

Base física: la luz es radiación electromagnética. Su distribución de longitudes de onda influye en el color percibido.

Luz que llega al ojo



Espectro visible · aprox. 380–780 nm

La radiación visible se compone de distintas longitudes de onda que la retina procesa como colores.

Respuesta del sistema visual

Tres tipos de conos responden a bandas solapadas del espectro.

Sus respuestas combinadas permiten distinguir la totalidad de los colores.



Conos S

Onda corta (Azul)



Conos M

Onda media (Verde)



Conos L

Onda larga (Rojo)

La percepción depende de la luz, los objetos, el observador y las condiciones de visión.

Para estudiar esta diapositiva

Los tres tipos de conos tienen sensibilidades solapadas, no detectores exclusivos de un único color. Conviene interpretar rojo, verde y azul como una simplificación didáctica de sus respuestas.

Teoría del color · CIE 1931 y gama reproducible

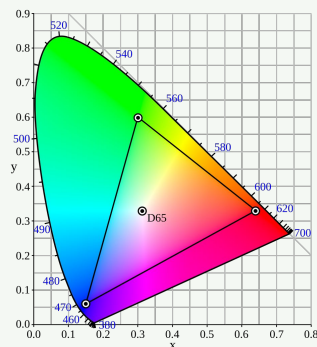
Colorimetría y Espacio CIE XYZ

- La colorimetría describe colores mediante valores triestímulo.
- CIE XYZ es un modelo de referencia estándar e independiente del dispositivo.

Diagrama de Cromaticidad (x, y)

- El diagrama x,y representa la cromaticidad aislando la luminancia.
- Permite analizar visualmente la pureza y el rango de color reproducible.

Diagrama CIE 1931



El triángulo delimita la gama de unos primarios RGB; no abarca todos los colores visibles.

[Referencia: CIE · Colorimetry](#)

Para estudiar esta diapositiva

El diagrama de cromaticidad separa información cromática y luminancia. El triángulo de un dispositivo muestra una gama limitada: cambiar la combinación de sus primarios no permite reproducir cualquier color visible.

Teoría del color · RGB y CMYK

RGB · Red, Green, Blue

Mezcla aditiva: combina luz roja, verde y azul.



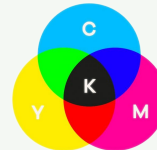
$R + G + B \rightarrow \text{blanco}$



Uso principal: Pantallas y proyectores.

CMYK · Cyan, Magenta, Yellow, Key

Mezcla sustractiva: las tintas absorben parte de la luz.



Uso principal: Impresión (K aporta negro).

Los modelos se relacionan mediante conversiones; la gama depende del dispositivo.

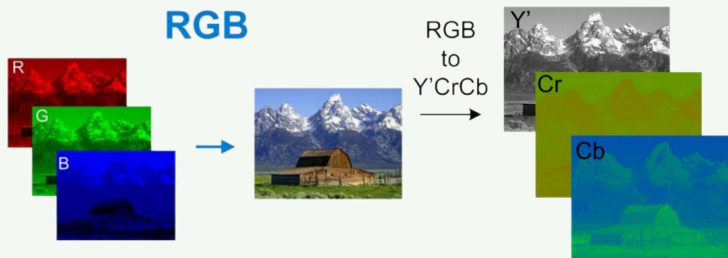
Para estudiar esta diapositiva

RGB combina luces y CMYK describe tintas que absorben luz. Por eso sus mezclas no siguen la misma intuición; una conversión entre ambos depende además de las características del dispositivo.

Teoría del color · YCbCr

Separa una componente de brillo (luminancia) de dos diferencias de color.

Descomposición de la imagen



Y' · luma

Aproxima el brillo de la imagen.

Cb · diferencia de azul

Información cromática azul.

Cr · diferencia de rojo

Información cromática roja.

Útil en JPEG y vídeo: permite tratar por separado el detalle de brillo y el de color.

Para estudiar esta diapositiva

Y' es luma y no debe confundirse exactamente con luminancia física. Separarla de Cb y Cr permite asignar distinta resolución a componentes que tienen diferente importancia perceptiva.

Teoría del color · HSL

Hue, Saturation, Lightness · matiz, saturación y claridad.

H · Matiz

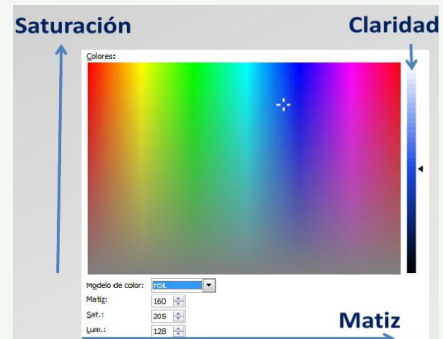
Familia de color: rojo, verde, azul...

S · Saturación

Del gris al color más saturado.

L · Claridad

Del negro al blanco, pasando por el color.

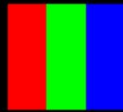


HSL facilita elegir colores. L es claridad del modelo, no luminancia física.

Para estudiar esta diapositiva

HSL reorganiza la descripción del color para facilitar su selección. Su componente L no es una medición física de luminancia ni garantiza saltos de brillo perceptualmente uniformes.

Teoría del color · demostración RGB

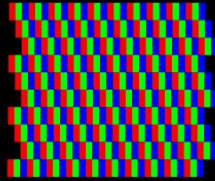


1 pixel
RGB

Para estudiar esta diapositiva

El esquema amplía un píxel RGB para distinguir sus contribuciones roja, verde y azul. La ampliación permite ver separadas componentes que, a la distancia habitual, se perciben de forma integrada.

Teoría del color · demostración RGB

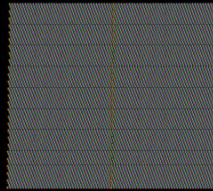


10 x 10 píxeles
RGB

Para estudiar esta diapositiva

La cuadrícula de 10×10 píxeles repite la estructura RGB. Compara con el píxel ampliado anterior: al reducir el tamaño aparente de cada componente, empieza a dominar la percepción del conjunto.

Teoría del color · demostración RGB

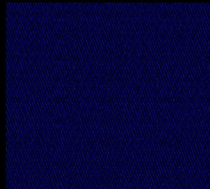


80 x 80 píxeles
RGB

Para estudiar esta diapositiva

En la retícula de 80×80 píxeles cuesta distinguir las componentes individuales. Esta demostración conecta la estructura RGB con la integración espacial estudiada antes.

Teoría del color · demostración RGB

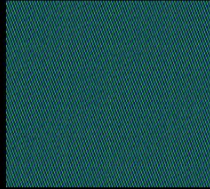


80 x 80 píxeles
Blue

Para estudiar esta diapositiva

Aquí se muestra la contribución azul de la retícula de 80×80. Compararla con las etapas siguientes ayuda a separar un canal del resultado compuesto.

Teoría del color · demostración RGB

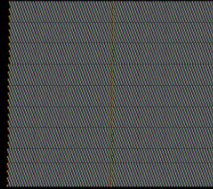


80 x 80 píxeles
Green + Blue

Para estudiar esta diapositiva

Se combinan las contribuciones verde y azul de la retícula. La mezcla aditiva cambia el color percibido; todavía falta la contribución roja que se incorpora después.

Teoría del color · demostración RGB



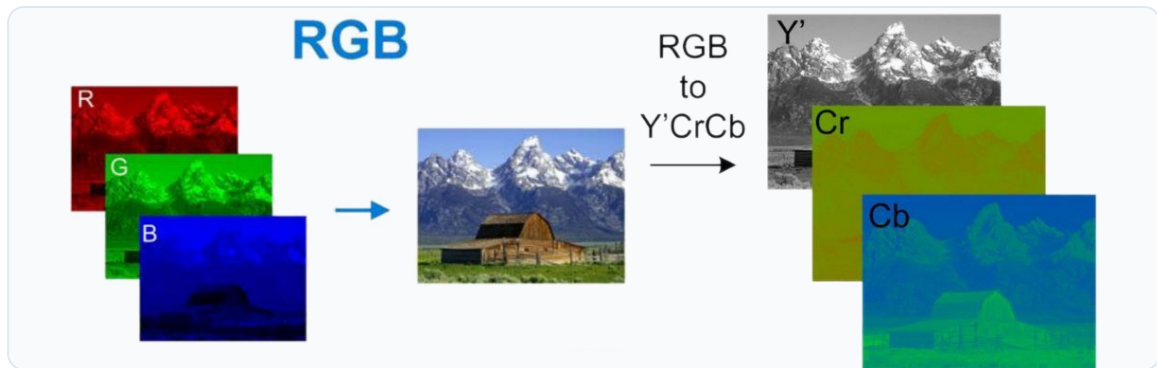
80 x 80 píxeles
Red + Green + Blue

Para estudiar esta diapositiva

Al reunir rojo, verde y azul se completa la demostración. La estructura de componentes sigue existiendo aunque, por integración espacial, percibamos principalmente el resultado conjunto.

Sensibilidad en la percepción de colores

Distinguimos mejor el detalle espacial de luminancia que los cambios finos de color.



💡 Por eso podemos reducir crominancia conservando más detalle de la componente Y'.

Para estudiar esta diapositiva

La menor sensibilidad al detalle cromático permite reducir datos de color en determinadas condiciones. La pérdida puede hacerse visible en bordes coloreados finos; no es una garantía de invisibilidad en toda imagen.

02 · Representación de imágenes

Mapa de bits · Matricial

Retícula de píxeles.

Cada posición almacena un valor de gris o de color.

FORMATOS: BMP · TIFF · JPEG · GIF · PNG

```
// Estructura de datos
Píxeles: [R,G,B] [R,G,B] ...
```

Formato vectorial · Objetos

Objetos geométricos con atributos:

coordenadas, tamaño, relleno, contorno, texto...

FORMATOS: SVG · AI · FIG · PDF*

```
<!-- Código SVG -->
<circle cx="50" r="20" .../>
```

* PDF puede combinar contenido vectorial, texto e imágenes de mapa de bits.

Para estudiar esta diapositiva

Un mapa de bits guarda muestras espaciales; un vectorial guarda descripciones geométricas. Algunos formatos, como PDF, pueden combinar ambas representaciones dentro del mismo documento.

Mapa de bits

La imagen se divide en una retícula de píxeles. Cada píxel tiene un valor de gris o un código de color.

Imagen original



Detalle ampliado (píxeles)



Al ampliar aparecen las celdas: aumentar el tamaño no recupera detalles que faltan.

Para estudiar esta diapositiva

Ampliar una imagen de mapa de bits hace visibles sus muestras o interpola valores entre ellas. No recupera automáticamente detalles que no fueron capturados en la resolución original.

Imágenes en formato vectorial

Colección de objetos

Líneas, curvas, polígonos, figuras y texto.

Atributos de cada objeto

Posición, dimensiones, escala, relleno, color y grosor del contorno.

Descripción geométrica

Se guarda cómo dibujar los objetos; se recalculan al cambiar de escala.



18 / 43

Para estudiar esta diapositiva

Un objeto vectorial se vuelve a dibujar a partir de sus atributos al cambiar la escala. Esto explica por qué líneas y curvas pueden conservar bordes nítidos al ampliarse.

Mapa de bits frente a formato vectorial

Vectorial

- Escala sin pixelarse y permite editar objetos por separado.

Adecuado para:

Logotipos, planos, esquemas e ilustración.

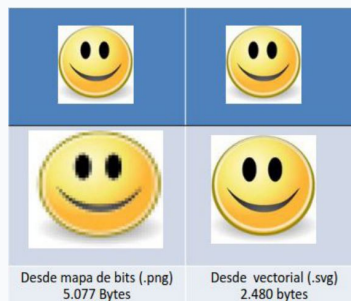
Mapa de bits

- Estructurado en retícula de píxeles individuales.

Adecuado para:

Fotografías y texturas de detalle complejo.

Comparativa de escalado y tamaño



En gráficos sencillos, el vector puede ocupar menos. No es una regla universal.

Para estudiar esta diapositiva

La mejor representación depende del contenido: fotografías y geometría simple plantean necesidades diferentes. Vectorial no significa siempre archivo pequeño; importan la complejidad y los elementos incorporados.

03 · Formatos de representación de imágenes con mapas de bits

Resolución

Anchura × altura

(número total de píxeles)

Profundidad de color

Bits por píxel

(niveles o colores posibles)

$$C = \frac{\text{anchura} \times \text{altura} \times \text{bits por píxel}}{8}$$

C: bytes de datos. El archivo puede añadir cabecera, paleta y relleno, o comprimirlos.

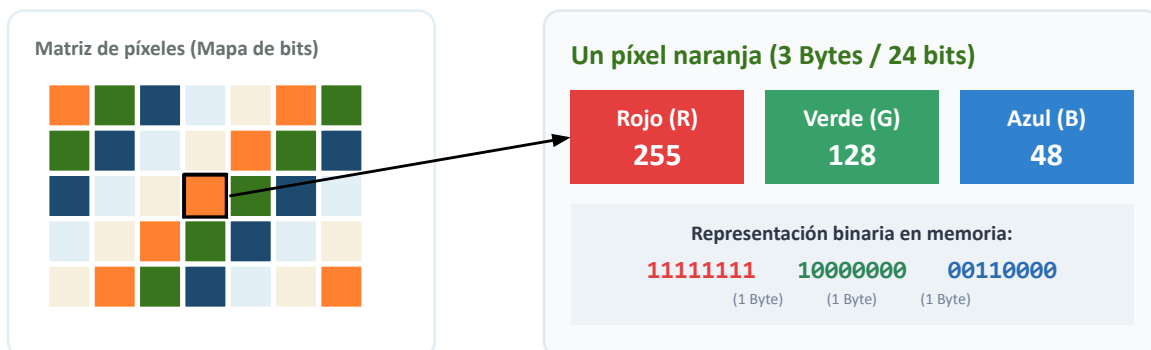
Para estudiar esta diapositiva

El tamaño básico se obtiene multiplicando anchura, altura y bits por píxel, y dividiendo entre ocho. Esta cuenta describe datos sin comprimir; cabeceras, paleta y rellenos pueden añadir espacio.

BMP (Windows BitMaP)

Ejemplo de color directo: cada píxel almacena sus componentes RGB.

Con 8 bits por componente: 24 bits = 3 bytes por píxel.



Formato sencillo y directo: sin compresión. El tamaño del archivo escala proporcionalmente al número de píxeles y a la profundidad de color.

Para estudiar esta diapositiva

En el ejemplo RGB de 24 bits, cada píxel almacena ocho bits para cada componente: tres bytes en total. La representación permite relacionar el color visible con los valores binarios de sus canales.

1. Tamaño de una imagen RGB

Datos de la imagen

Una imagen tiene **10 × 15 píxeles** y se representa en **RGB**, con **24 bits** por píxel.

10 × 15 px 24 bits / px

Cuestión

¿Cuántos bytes ocupan sus datos sin comprimir?

No incluyas cabecera ni relleno.

a) 24 B

b) 3.600 B

c) 450 B

d) 150 B

Para estudiar esta diapositiva

$10 \times 15 = 150$ píxeles y 24 bits equivalen a tres bytes por píxel. El enunciado excluye cabeceras y relleno, así que solo cuenta los datos RGB.

Simplemente calculamos su capacidad

Paso 1: Fórmula General

$$C = \frac{\text{anchura} \times \text{altura} \times \text{bits/píxel}}{8}$$

Paso 2: Sustitución y Cálculo

$$C = \frac{10 \times 15 \times 24}{8}$$

450 B de datos de imagen · respuesta c)

Para estudiar esta diapositiva

La división entre ocho convierte bits a bytes: $150 \times 24 / 8 = 450$. Mantener escritas las unidades ayuda a evitar responder 3.600 bytes cuando se han calculado bits.

2. Tamaño de una imagen en escala de grises

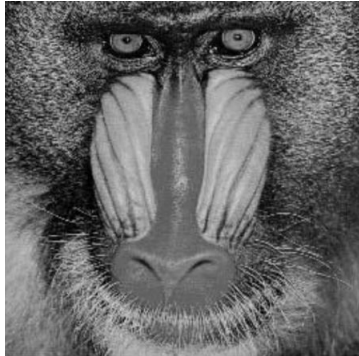


Imagen en escala de grises

Datos de la imagen

Resolución: **256 × 256 píxeles**

Profundidad: **8 bits por píxel (1 B/px)**

Cuestión

¿Cuántos KiB ocupan los datos sin compresión, cabecera ni paleta?

Nota: Expresa el resultado final en Kibibytes (KiB).

Para estudiar esta diapositiva

En escala de grises con ocho bits, cada píxel ocupa un byte. El resultado pedido en KiB requiere dividir los bytes entre 1024, no entre 1000.

Calculamos los bytes y convertimos a KiB

Paso 1: Cálculo en Bytes (B)

$$C = \frac{256 \times 256 \times 8}{8}$$
$$= 65.536 \text{ B}$$

Paso 2: Conversión a Kibibytes (KiB)

$$C = \frac{65.536 \text{ B}}{1.024 \text{ B/KiB}}$$
$$= 64 \text{ KiB}$$

64 KiB de datos de imagen

Para estudiar esta diapositiva

256×256 produce 65.536 píxeles y otros tantos bytes. Al dividir entre 1024 se obtienen 64 KiB bajo las exclusiones del enunciado.

TIFF (Tagged Image File Format)

Versatilidad

Formato flexible para raster

Soporta múltiples modos de color y resoluciones:

- RGB y CMYK
- Escala de grises
- Distintas profundidades de color

Compresión

Opción sin pérdida

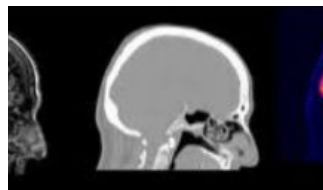
Permite guardar datos puros o comprimidos:

- Sin compresión
- LZW o ZIP sin pérdida

Nota: Puede contener JPEG, por lo que «TIFF» no garantiza siempre ausencia de pérdida.

Uso Profesional

Fotografía, impresión, archivo e imagen científica.



Para estudiar esta diapositiva

TIFF admite configuraciones variadas. Para saber si una imagen concreta usa pérdidas hay que examinar su codificación, no deducirlo únicamente de la extensión del archivo.

PNG (Portable Network Graphics)

Compresión

Sin pérdida

Recuperamos exactamente los datos codificados originales.

- Sin degradación de calidad
- Ideal para edición repetida

Canal Alfa y Color

RGB, Grises o Paleta

Permite transparencia con distintos grados de opacidad.



Casos de Uso

Gráficos e Interfaces

Especialmente útil en:

- Capturas de pantalla
- Interfaces de usuario
- Diagramas e ilustraciones
- Imágenes con texto y bordes nítidos

Referencia: [Especificación PNG \(W3C\)](#)

Para estudiar esta diapositiva

PNG conserva los datos codificados sin pérdidas y admite transparencia. Eso no recupera información eliminada antes de crear el PNG, por ejemplo por una reducción previa de colores.

JPEG (Joint Photographic Experts Group)

Uso y Tipo

Fotografías y Pérdida

Características principales del formato:

- Muy utilizado para fotografías
- Compresión habitual con pérdida
- Altas tasas de reducción de datos

Mecanismo

Percepción Visual

Optimizaciones aplicadas:

- Aprovecha la menor sensibilidad al detalle de color
- Descarta parte de la información fina

Ajuste de Calidad

Equilibra el tamaño y la degradación visible.



Nota: Al comprimir más pueden aparecer bloques, halos y pérdida de detalle.

[Abrir recurso interactivo: JPEG paso a paso](#)

28 / 43

Para estudiar esta diapositiva

Una compresión JPEG más intensa suele reducir tamaño a cambio de más degradación. Los artefactos pueden apreciarse especialmente en texto, bordes nítidos y áreas de poco detalle.

JPEG · Luminancia y Crominancia

Componente Y'

Luminancia (Brillo)

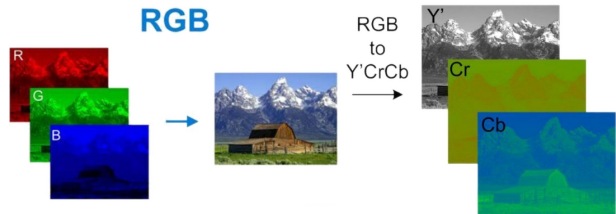
Conserva todo el detalle visual e intensidad de la imagen. Es el componente crítico para la percepción humana.

Componentes Cb y Cr

Crominancia (Color)

Representan las diferencias de color (azul y rojo). Pueden muestrearse a menor resolución sin pérdida de calidad visible.

Separación de Canales: RGB $\rightarrow$ Y'CbCr



Principio clave: La visión humana es más sensible a variaciones de brillo (Y') que a diferencias de color (Cb, Cr).

Para estudiar esta diapositiva

Separar Y', Cb y Cr facilita conservar más resolución de brillo que de color. El submuestreo sí elimina información cromática y puede producir diferencias visibles en determinadas imágenes.

JPEG · Submuestreo de crominancia

En una región de 4×2 píxeles, ¿cuántos valores conservamos?

4:4:4

Y'  8 valores

Cb  8 valores

Cr  8 valores

Total conservado: $8 + 8 + 8 = 24$

4:2:2

Y'  8 valores

Cb  4 valores

Cr  4 valores

Total conservado: $8 + 4 + 4 = 16$

4:2:0

Y'  8 valores

Cb  2 valores

Cr  2 valores

Total conservado: $8 + 2 + 2 = 12$

Nota: Las celdas de crominancia cubren regiones mayores: cada valor se comparte entre píxeles.

Para estudiar esta diapositiva

Cuenta valores de componentes, no solo píxeles. En el bloque mostrado, 4:4:4 conserva 24 valores, 4:2:2 conserva 16 y 4:2:0 conserva 12, antes de otras etapas de compresión.

JPEG · Del submuestreo al archivo comprimido

1

Separar componentes

RGB $\rightarrow$ Y', Cb y Cr

Submuestreo opcional del color para reducir ancho de banda perceptivo.

2

Transformar bloques

Bloques de 8×8

Una transformación espacial (DCT) describe las variaciones de frecuencia dentro de los bloques.

3

Cuantificar

Pérdida con control

Se redondean coeficientes. Aquí es exactamente donde se descarta la información menos perceptible.

4

Codificar

Compresión final

Se aprovechan los ceros consecutivos y secuencias repetidas para escribir menos bits de salida.

Nota importante: El factor final de compresión depende de la complejidad de la imagen y del codificador, no solo del submuestreo.

→ [Abrir recurso interactivo: JPEG paso a paso](#)

Para estudiar esta diapositiva

La transformación cambia cómo se describe el bloque; la cuantificación redondea coeficientes y pierde información. Si se aplica submuestreo antes, esa etapa también ha reducido información cromática.

3. Compresión por submuestreo de color

Planteamiento del problema

Formato RGB original

Usa **24 bits por píxel**. Consideramos bloques en cuadrados de **5 × 5 píxeles** (25 píxeles).

Submuestreo Y'CbCr

Por cada cuadrado se guardan **25 valores Y'**, **1 Cb** y **1 Cr**.

Profundidad de color

Cada valor de componente usa **8 bits**.

Objetivo del ejercicio

Calcula el factor de reducción:

FACTOR DE COMPRESIÓN

Tamaño inicial

Tamaño final

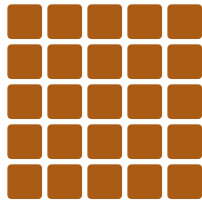
Nota: Modelo simplificado de submuestreo; no calcula por sí solo el tamaño de un JPEG.

Para estudiar esta diapositiva

Este modelo guarda 25 valores de Y' y solo dos de crominancia por bloque. Es una simplificación para contar datos, no una predicción completa del tamaño de un JPEG real.

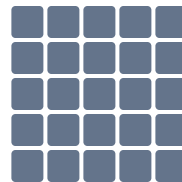
i) Contamos datos por bloque de 25 píxeles

RGB original



$$C_1 = 25 \times (8 + 8 + 8) \\ = 600 \text{ bits}$$

Modelo Y'CbCr



25 × Y'

Cb

Cr

$$C_2 = (25 + 1 + 1) \times 8 \\ = 216 \text{ bits}$$

Para estudiar esta diapositiva

RGB almacena 75 componentes de ocho bits en 25 píxeles; el modelo reducido almacena 27. Por eso se comparan 600 y 216 bits por bloque.

ii) Calculamos el factor de compresión

1. Fórmula general

$$fc = Ca / Cd$$

Ca: Tamaño antes (original)

Cd: Tamaño después (comprimido)

2. Sustitución de datos

$$fc = 600 \text{ bits} / 216 \text{ bits}$$

600 bits: Bloque RGB original

216 bits: Bloque Y'CbCr comprimido

3. Factor obtenido

$$f \approx 2,78 \text{ a } 1$$

El archivo comprimido es **2,78 veces**
más pequeño que el original.

Conclusión: Reducción global a más de la mitad del tamaño de datos (Ahorro de ~64% de espacio).

Para estudiar esta diapositiva

$600/216 \approx 2,78$ es el factor de reducción. La fracción que queda es $216/600 = 0,36$ y el ahorro es 64 %; son tres formas relacionadas de expresar el resultado.

GIF (Graphics Interchange Format)

GIF guarda una paleta RGB y un índice por píxel.

1. Imagen indexada



Cada píxel hace referencia a una posición de la paleta.

2. Paleta de colores



256 entradas
(Índices del 0 al 255)

3. Mapeo de píxel

Índice 2
00000010₂



Color RGB
(147, 154, 162)

El índice direcciona al color final en la tabla

256 colores → 2⁸ índices posibles → 8 bits por píxel, más la paleta.

Para estudiar esta diapositiva

En una imagen indexada, el byte del píxel apunta a una entrada de la paleta. No contiene directamente tres intensidades RGB; para recuperar el color hay que consultar la tabla.

GIF · Compresión, Transparencia y Animación

1. Compresión LZW

Sin pérdida: Comprime los índices manteniendo la información exacta.

Paleta: Reducir una foto a 256 colores sí puede perder fidelidad.

2. Transparencia

Índice reservado: Un código de la paleta se asigna como transparente.

Integración: Permite superponer la imagen sobre cualquier fondo.

3. Animación

Secuencia: Guarda múltiples fotogramas dentro del mismo archivo.

Temporización: Define la duración de exposición de cada cuadro.

Mapeo de la Animación

Los fotogramas se muestran de forma sucesiva en bucle, asignando un intervalo de tiempo específico a cada uno.



Para estudiar esta diapositiva

La compresión de los índices puede ser sin pérdidas aunque convertir una fotografía a una paleta limitada haya perdido colores previamente. Son etapas diferentes que conviene evaluar por separado.

GIF · Selección de la paleta de colores

Ajuste de algoritmos, número de colores y tramado para la optimización de la imagen.

1. Algoritmo de paleta

Selección del método (Selectiva, Adaptativa, Perceptual) para identificar los colores más representativos.

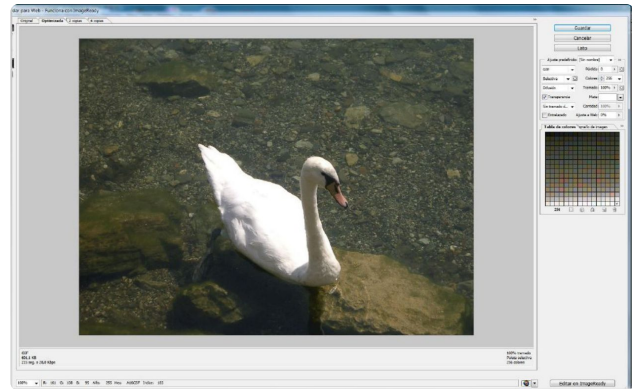
2. Cantidad de colores

Reducción de la tabla (hasta 256 entradas). Menos colores reduce drásticamente el peso del archivo.

3. Tramado (Dithering)

Mezcla visual de píxeles adyacentes para simular degradados y matices ausentes en la paleta.

Ajustes de optimización y tabla de colores



Para estudiar esta diapositiva

El tramado distribuye colores disponibles para simular visualmente otros tonos. Reducir la paleta puede ahorrar datos, pero el tamaño final también depende de la imagen y de la compresión.

GIF · La paleta también ocupa espacio

La paleta se guarda una sola vez para la imagen. Cada entrada contiene un color **RGB de 24 bits**.

1. Cálculo por cada color

R: 8 bits + G: 8 bits + B: 8 bits = **24 bits**

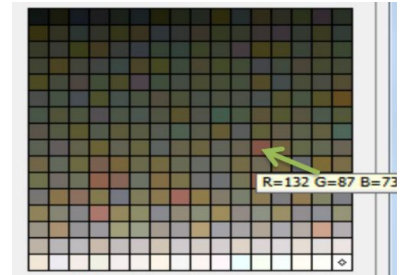
24 bits / 8 = **3 bytes / color**

2. Tamaño total de la paleta (256 colores)

256 colores × 3 B/color

= 768 B

Tabla de colores de la imagen



Paleta de 256 colores del original

Para estudiar esta diapositiva

Cada entrada RGB de 24 bits ocupa tres bytes. La paleta completa de 256 entradas suma 768 bytes y se cuenta una sola vez, no una vez por píxel.

04 · Compresión y tamaños de archivo

Una misma imagen: distintos tamaños de archivo

Ejemplo original: cisne de 1.024 × 768 píxeles, RGB de 24 bits (mediciones con Photoshop 7).

BMP / TIFF

Ejemplo original sin compresión

≈ **3 MB**

Sin comprimir

Máxima calidad, mayor ocupación en disco.

JPEG

Compresión con pérdida

Alta: **523 KB**

Media: **81,33 KB**

Baja: **43,16 KB**

GIF

Paleta de colores indexada

256 colores:

550 KB

128 colores:

461 KB

PNG

Sin pérdida / Paleta

Modo RGB (Truecolor):

1,15 MB

Paleta 256 colores:

530 KB

Datos RGB teóricos: 2.359.296 B = 2,25 MiB. Las cifras de archivo proceden del ejemplo original.

Para estudiar esta diapositiva

Las cifras son mediciones de un ejemplo concreto y no porcentajes universales de los formatos. Para comparar limpiamente, distingue datos RGB teóricos y tamaño real del archivo con sus ajustes.

4. Representación RGB frente a una paleta

Se tiene una imagen de **1600 × 900 píxeles** de resolución, originalmente en formato **BMP (truecolor)**.

Si, en su lugar, se utilizase el formato **GIF con una paleta de 256 colores**:

a) Tabla de colores

¿Cuánto ocuparía la tabla de colores de la paleta?

b) Conjunto de píxeles

¿Cuánto ocuparía la información asociada a la propia imagen (conjunto de píxeles)?

c) Factor de compresión

¿Cuál sería el factor de compresión del fichero BMP (truecolor) al representarlo en GIF?

Modelo del ejercicio: RGB de 24 bits; sin cabeceras ni compresión LZW.

Para estudiar esta diapositiva

Se compara RGB de tres bytes por píxel con índices de un byte y una paleta. El ejercicio excluye cabeceras y LZW: calcula una representación simplificada, no un GIF comprimido real.

4.a) Capacidad de la paleta

La paleta contiene 256 colores. Cada color necesita un byte para R, otro para G y otro para B.

R · 1 byte

G · 1 byte

B · 1 byte

1 color = 3 bytes

256 colores × 3 bytes/color = 768 bytes

$768 / 1024 = 0,75$ KiB. La paleta no se repite por cada píxel.

Para estudiar esta diapositiva

La resolución de la imagen no interviene en este apartado: la paleta depende del número de entradas y de los bytes por entrada. $256 \times 3 = 768$ bytes.

4.b) Capacidad de los índices y del conjunto

Cada píxel guarda el índice de uno de los 256 colores.
Como $2^8 = 256$, basta un índice de 8 bits = 1 byte.

$$1.600 \times 900 = 1.440.000 \text{ píxeles}$$

$$1.440.000 \text{ píxeles} \times 1 \text{ B/píxel} = 1.440.000 \text{ B}$$

A los índices hay que añadir el tamaño de la paleta una sola vez (768 B):

$$1.440.000 + 768 = 1.440.768 \text{ B}$$

Para estudiar esta diapositiva

Los 1.440.000 índices ocupan 1.440.000 bytes. A ese total se añaden los 768 bytes de la paleta una sola vez para obtener el conjunto representado.

4.c) Factor de compresión

Antes

cada píxel guarda R, G y B → 3 bytes.

$$1.600 \times 900 \times 3 = 4.320.000 \text{ B}$$

Después

1 byte por píxel + la paleta completa.

$$1.440.000 + 768 = 1.440.768 \text{ B}$$

Factor = tamaño original / tamaño indexado

$$4.320.000 / 1.440.768 \approx 2,998 \approx 3$$

La representación ocupa aproximadamente un tercio del tamaño original.

Para estudiar esta diapositiva

El factor queda ligeramente por debajo de tres porque la paleta añade un coste fijo. Cuanto mayor sea la imagen, menor será el peso relativo de ese coste en este modelo.

Tema 2 · Nuestro recorrido

L2.0 Sistemas de numeración usuales en informática

L2.1 Nociones básicas sobre representación de la información

L2.2 Representación de textos

L2.3 Representación de sonidos

L2.4 Representación de imágenes

L2.5 Representación de vídeo

L2.6 Representación de números enteros

L2.7 Representación de números reales

Para estudiar esta diapositiva

El vídeo combina imágenes ordenadas en el tiempo. Reutilizaremos el tamaño de una imagen y añadiremos frecuencia de fotogramas, duración y dependencias entre imágenes.

L2.5

Representación de vídeo

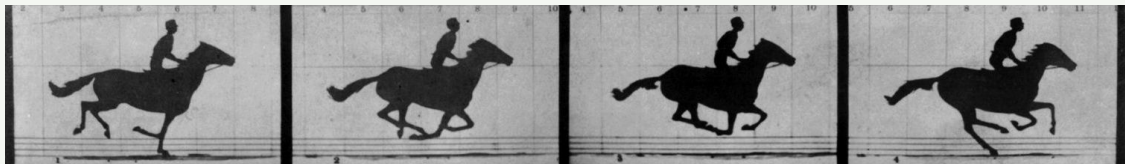
01 Representar vídeo Ejercicios de tamaño y frecuencia

Para estudiar esta diapositiva

Conviene separar el cálculo sin compresión de los modelos que reutilizan información entre fotogramas. Un archivo audiovisual también puede incluir audio y otras pistas.

01 · Del movimiento continuo a los fotogramas

Un movimiento contiene infinitos instantes: **no podemos guardarlos todos.**



E. Muybridge · The Horse in Motion (1878) · dominio público

Muestreamos el tiempo: guardamos una secuencia finita de imágenes.

- **Integración temporal:** percibimos movimiento al presentar la secuencia rápidamente.
- **Frecuencia (fps):** fotogramas por segundo que indican la cadencia de la reproducción.

Para estudiar esta diapositiva

Una secuencia conserva estados discretos de un movimiento continuo. Los fps indican cuántos se presentan cada segundo; no son el número de píxeles ni la profundidad de color.

Caudal de datos y capacidad de memoria

VÍDEO SIN COMPRIMIR · multiplicamos las tres cantidades

$$R_{\text{bps}} = f_{\text{fps}} \times n_{\text{píxeles/imagen}} \times n_{\text{bits/píxel}}$$

f · fotogramas/s

Cuántas imágenes por segundo

n · píxeles/imagen

Anchura × altura de cada imagen

n · bits/píxel

Profundidad de color

$$C_{\text{bytes}} = R_{\text{bps}} \times t_{\text{segundos}} / 8$$

R es el caudal (bits/s); C es la memoria ocupada durante t segundos.

Ejemplo: $25 \times (640 \times 480) \times 24 = 184.320.000 \text{ bit/s}$.

En 60 s: $184.320.000 \times 60 / 8 = 1.382.400.000 \text{ B} \approx 1,38 \text{ GB}$.

Para estudiar esta diapositiva

Multiplicar fps por píxeles por imagen y bits por píxel produce bits por segundo. Para obtener bytes de una duración concreta, multiplica por segundos y divide entre ocho.

Compresión: GOP · Grupo de Imágenes (Group of Pictures)

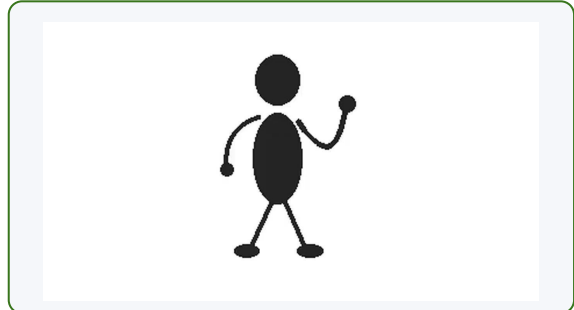
Un GOP organiza varios fotogramas y sus referencias para comprimirlos aprovechando lo que tienen en común.

¿Qué pretende?

Evitar guardar una y otra vez la información que se repite.

Ejemplo práctico

En este vídeo, el cuerpo permanece igual y solo cambia el brazo.



La información compartida se reutiliza; los cambios permiten reconstruir cada imagen.

Para estudiar esta diapositiva

La compresión temporal aprovecha relaciones entre imágenes cercanas. El movimiento no obliga a almacenar cada imagen completa, pero la reconstrucción necesita las referencias y la información de cambio correspondiente.

GOP: Fotogramas I, P y B — Referencias y Orden

I (Intra) Imagen autónoma (sin dependencias) **P (Predictiva)** Referencia anterior **B (Bidireccional)** Referencia anterior y posterior

1. ORDEN DE VISUALIZACIÓN (Muestra final en pantalla para el usuario)



Principio de compresión temporal: En los fotogramas P y B solo se almacena la zona que cambia respecto a sus fotogramas de referencia. El códec únicamente guarda vectores de movimiento y residuos de información.

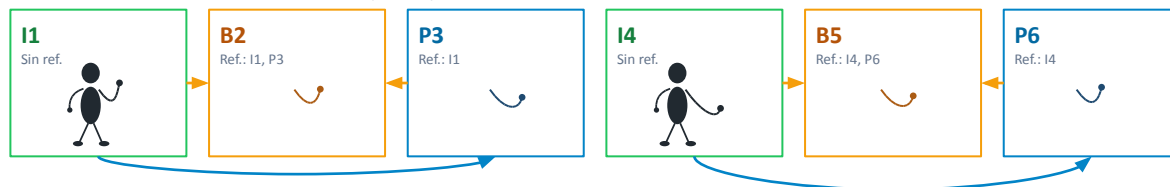
Para estudiar esta diapositiva

Los fotogramas predictivos se reconstruyen a partir de referencias y datos adicionales. La idea de «guardar solo lo que cambia» es una simplificación: los códecs también codifican modos, residuos y, cuando procede, bloques intra.

GOP: Fotogramas I, P y B — Referencias y Orden

I (Intra) Imagen autónoma (sin dependencias) **P (Predictiva)** Referencia anterior **B (Bidireccional)** Referencia anterior y posterior

1. ORDEN DE VISUALIZACIÓN (Muestra final en pantalla para el usuario)



2. ORDEN DE DECODIFICACIÓN (Procesamiento y flujo de datos en el códec)



Principio de compresión temporal: En los fotogramas P y B solo se almacena la zona que cambia respecto a sus fotogramas de referencia. El códec únicamente guarda vectores de movimiento y residuos de información.

Para estudiar esta diapositiva

Una imagen debe disponer de sus referencias antes de decodificarse. Por eso el orden de procesamiento puede diferir del de visualización, aunque el espectador vea la secuencia temporal correcta.

Cuando se daña un fotograma de referencia



I97 · La referencia se daña

Se pierden paquetes que contienen datos del fotograma I. Su reconstrucción queda incompleta o alterada.

P140 · El error se propaga

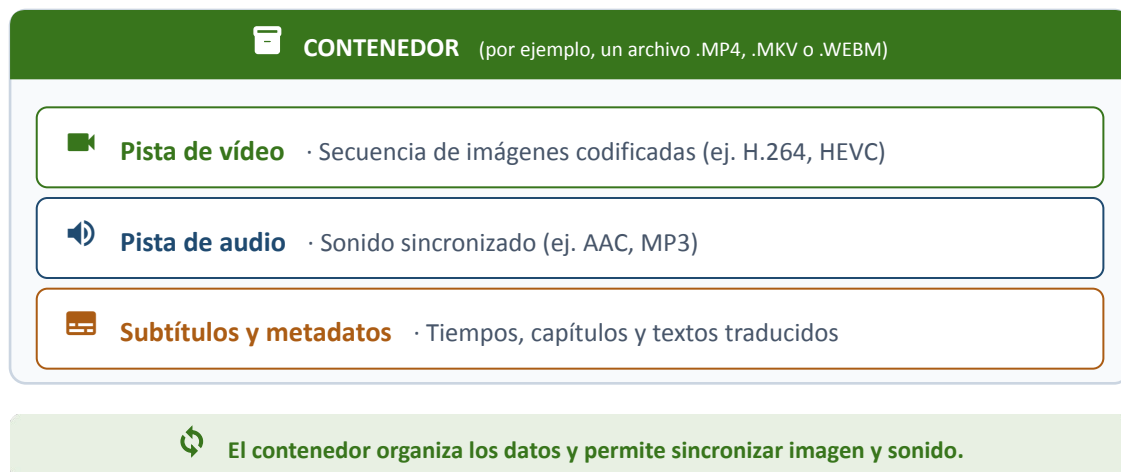
Los fotogramas que dependen de esa referencia heredan parte del daño: el error persiste en imágenes posteriores.

[Vídeo a 1 Mbit/s. Moorthy et al. \(2010\). Wireless Video Quality Assessment, fig. 3.](#)

Para estudiar esta diapositiva

Si una referencia queda dañada, las imágenes que dependen de ella pueden heredar errores. La duración del efecto depende de las dependencias y de los mecanismos de recuperación del sistema.

El contenedor reúne las pistas del vídeo



Para estudiar esta diapositiva

Un contenedor reúne pistas y datos de sincronización. Vídeo, audio y subtítulos pueden usar codificaciones distintas aunque se transporten dentro de un único archivo.

Contenedor y códec: funciones distintas



Contenedor

Organiza las pistas del archivo.

Ejemplos: **MP4**, **Matroska**, **WebM**.



Códec (codificador/decodificador)

Comprime y reconstruye la señal.

Ejemplos de vídeo: **H.264**, **AV2**, **VP9**



Un mismo contenedor puede admitir diferentes códecs.



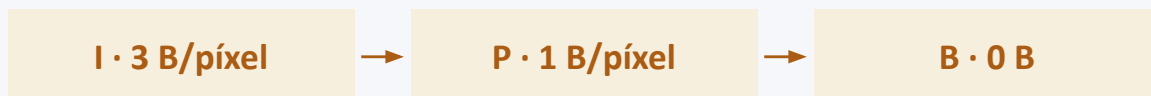
Para reproducir el archivo deben ser compatibles tanto el contenedor como sus códecs.

Para estudiar esta diapositiva

El contenedor organiza y el códec codifica o decodifica. Para reproducir un archivo hace falta comprender ambos; la extensión por sí sola no garantiza que estén disponibles los decodificadores necesarios.

1. ¿Cuánto ocupa el vídeo?

Un vídeo de 5 minutos contiene 9.000 fotogramas de 1,4 megapíxeles. Para reducir los datos se agrupan de tres en tres: el primero se guarda completo, el segundo mediante diferencias y el tercero se reconstruye.



¿Cuántos GB ocupan los datos del vídeo?

Modelo del ejercicio: grupos I–P–B; B sin datos, sin audio ni metadatos.

Usa unidades decimales: 1 megapíxel = 10^6 píxeles; 1 GB = 10^9 B.

Para estudiar esta diapositiva

Usa exactamente el modelo del enunciado: B no aporta bytes, aunque sí cuenta como imagen. Es una hipótesis didáctica y no una propiedad general de todos los fotogramas B reales.

Contamos grupos y calculamos su coste

Cada grupo contiene 3 imágenes → $9.000 / 3 = 3.000$ grupos

I · imagen completa	P · diferencias	B · reconstruida
$1.400.000 \times 3 \text{ B}$ = 4,2 MB	$1.400.000 \times 1 \text{ B}$ = 1,4 MB	Sin datos almacenados = 0 MB

Un grupo cuesta $4,2 + 1,4 + 0 = 5,6 \text{ MB}$

$$3.000 \text{ grupos} \times 5,6 \text{ MB/grupo} = 16.800 \text{ MB} = \mathbf{16,8 \text{ GB}}$$

Para estudiar esta diapositiva

Cada grupo cuesta 5,6 MB y hay 3.000 grupos. El producto da 16,8 GB con unidades decimales; no añadas audio ni metadatos porque el modelo los excluye.

2. ¿A cuántos fotogramas por segundo se reproduce?

Un vídeo de 10 minutos ocupa 16,1 GB. Sus fotogramas tienen 1.600×900 píxeles y se organizan en grupos de tres (I-P-B). I guarda la imagen con 16 bits/píxel; P, las diferencias con 8 bits/píxel; B se reconstruye al reproducir el vídeo y no almacena datos.

DURACIÓN Y TAMAÑO

10 min
16,1 GB

RESOLUCIÓN

1.600 × 900
píxeles por imagen

ESTRUCTURA

Grupos de 3
(I - P - B)

CODIFICACIÓN

I: 16 bits/píxel
P: 8 bits/píxel
B: Reconstruye (0 B)

Calcula la frecuencia de fotogramas del vídeo, en fps.

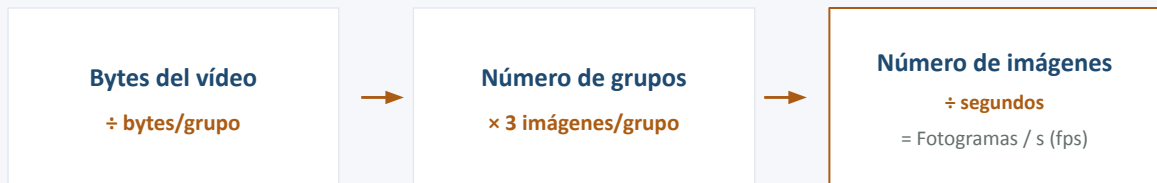
- Supón grupos completos, sin audio ni metadatos. 1 GB = 10^9 B.
- El tamaño está redondeado; obtendremos una frecuencia aproximada.

Para estudiar esta diapositiva

El tamaño se da redondeado, así que la frecuencia final será aproximada. Primero calcula el coste del grupo I-P-B y después deduce cuántas imágenes representa el archivo.

Planteamiento de la resolución del problema

Sabemos cuántos bytes ocupa el vídeo. Para contar sus imágenes, primero debemos averiguar cuántos bytes cuesta cada grupo.



DATOS DE PARTIDA DEL PROBLEMA:

- Capacidad: **16,1 GB** = 16.100.000.000 B
- Duración: **10 min** × 60 s/min = 600 s

Para estudiar esta diapositiva

Sigue las unidades: bytes divididos por bytes/grupo dan grupos; multiplicar por imágenes/grupo da imágenes. Al dividir entre segundos se obtiene la frecuencia en fps.

i) ¿Cuántos bytes cuesta un grupo de tres?

Primero contamos píxeles: $1.600 \times 900 = 1.440.000$ píxeles por imagen.

$$\begin{aligned} & \mathbf{I \cdot 16 \text{ bits/píxel}} \\ & 16 / 8 = 2 \text{ B/píxel} \\ & 1.440.000 \times 2 \\ & \mathbf{= 2.880.000 \text{ B}} \end{aligned}$$

$$\begin{aligned} & \mathbf{P \cdot 8 \text{ bits/píxel}} \\ & 8 / 8 = 1 \text{ B/píxel} \\ & 1.440.000 \times 1 \\ & \mathbf{= 1.440.000 \text{ B}} \end{aligned}$$

$$\begin{aligned} & \mathbf{B \cdot \text{sin datos}} \\ & \text{Se reconstruye} \\ & \text{al reproducir.} \\ & \mathbf{= 0 \text{ B}} \end{aligned}$$

$$\mathbf{\text{Un grupo: } 2.880.000 + 1.440.000 = 4.320.000 \text{ B}}$$

Para estudiar esta diapositiva

16 bits por píxel equivalen a dos bytes y ocho bits equivalen a uno. El coste del grupo es 4.320.000 bytes bajo la hipótesis de que B no almacena datos.

ii) Del número de grupos al número de imágenes

PASO 1

Dividimos el tamaño total del vídeo entre lo que ocupa un grupo:

$$16.100.000.000 \text{ B} \div 4.320.000 \text{ B/grupo} \\ \approx 3.726,85 \text{ grupos}$$



PASO 2

Cada grupo permite reproducir 3 imágenes (incluida la B).

Aunque B no guarde datos, cuenta como fotograma.

$$\text{Número de grupos} \times 3 \\ \approx 11.180,56 \text{ fotogramas}$$

* Los decimales proceden del tamaño redondeado: no existen fracciones de fotograma.

Para estudiar esta diapositiva

Aunque B no cueste bytes en este modelo, cuenta entre las tres imágenes del grupo. Los resultados fraccionarios intermedios reflejan el tamaño redondeado del archivo, no imágenes físicamente fraccionadas.

iii) ¿Cuántos fotogramas corresponden a un segundo?

El vídeo dura **600 s** y contiene aproximadamente **11.180,56 fotogramas**.

TODO EL VÍDEO

11.180,56 fotogramas

en 600 segundos

÷ 600

EN UN SEGUNDO

18,63 fotogramas

en 1 segundo

Dividimos las imágenes y el tiempo entre 600: obtenemos imágenes por segundo.

$$f = 11.180,56 \text{ fotogramas} / 600 \text{ s} \approx 18,63 \text{ fps}$$

Para estudiar esta diapositiva

La frecuencia se obtiene dividiendo las imágenes totales entre 600 segundos. Aproximadamente 18,63 fps es la consecuencia de los datos y supuestos del ejercicio.

Tema 2 · Nuestro recorrido

L2.0 Sistemas de numeración usuales en informática

L2.1 Nociones básicas sobre representación de la información

L2.2 Representación de textos

L2.3 Representación de sonidos

L2.4 Representación de imágenes

L2.5 Representación de vídeo

L2.6 Representación de números enteros

L2.7 Representación de números reales

Para estudiar esta diapositiva

Volvemos a los números para estudiar su almacenamiento con un ancho fijo. La dificultad nueva es representar el signo y determinar qué sucede cuando una operación sale del intervalo disponible.

L2.6

Representación de números enteros

01 Interpretar bits 02 SM 03 CA1 04 CA2
05 Sesgado 06 BCD y límites

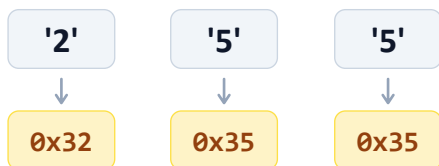
Para estudiar esta diapositiva

Las convenciones de enteros asignan significados distintos a patrones iguales.
Compara siempre número de bits, regla de codificación, intervalo y tratamiento del cero.

Guardar un número no es guardar su texto

Texto «255»

Codificación de caracteres (ASCII / Hex)



3 bytes (24 bits) · ASCII Hexadecimal

Cada carácter requiere un byte independiente

Entero 255 sin signo

Representación numérico-binaria directa

REGISTRO DE 8 BITS (1 BYTE)



1 byte (8 bits) · Binario natural

Eficiencia x3 en espacio y listo para operar directamente

255 + 42 = 297: sumamos valores, no sus caracteres.

El procesamiento numérico opera sobre la representación binaria.

Para estudiar esta diapositiva

El texto «255» necesita tres caracteres ASCII, mientras que el entero sin signo 255 cabe en ocho bits. Son representaciones distintas; la ventaja de tamaño de este ejemplo no es una proporción universal.

Ya sabemos escribir un entero en distintas bases

Un mismo valor; distintas escrituras numéricas.

Representación de 28_{10} en un registro de 8 bits

Suma de pesos posicionales: $16 + 8 + 4 = 28$



$2^7 (128) 2^6 (64) 2^5 (32) 2^4 (16) 2^3 (8) 2^2 (4) 2^1 (2) 2^0 (1)$

$$28_{10} = 34_8 = 1C_{16}$$

Equivalencia directa en bases decimal, octal y hex

Enteros sin signo en C

Tipo entero de ancho fijo de 8 bits

```
uint8_t x = 5;
```

- Bits en memoria: **00000101**
- Cabecera: Definido en `<stdint.h>`
- Rango de valores: 0 a 255 ($2^8 - 1$)

En binario natural:

El valor numérico es la suma de los pesos 2^i correspondientes a los bits con valor 1.

Para estudiar esta diapositiva

$28=16+8+4$, de modo que su patrón de ocho bits es 00011100. Un tipo de ancho fijo permite especificar el rango sin depender de que int tenga un tamaño determinado.

¿Y cómo guardamos -28 en ocho bits?

Con ocho posiciones solo podemos escribir ceros y unos.

?	?	?	?	?	?	?	?
---	---	---	---	---	---	---	---

No existe una casilla para escribir «-».

Los circuitos digitales solo reconocen dos estados físicos posibles (0 y 1).

Necesitamos acordar una convención:

Los bits por sí solos no indican cómo interpretarlos.

Para estudiar esta diapositiva

El signo menos no se guarda como un símbolo adicional en estas representaciones de ancho fijo. Debe incorporarse a la interpretación de los mismos ocho bits disponibles.

Cómo representamos los números enteros



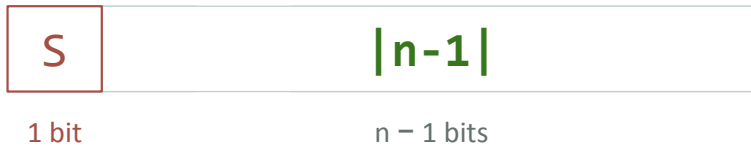
A continuación: profundizamos en la representación por **signo y magnitud**

Para estudiar esta diapositiva

El mapa separa enteros sin signo y convenciones con signo. BCD, por su parte, representa cifras decimales individualmente; no debe confundirse con convertir todo el valor a binario natural.

Signo y magnitud (SM)

- El bit más significativo indica el signo (**S**)
- Los $n-1$ bits restantes indican el valor absoluto (**$|n-1|$**)



$S = 0 \rightarrow$ positivo o + $S = 1 \rightarrow$ negativo o -

Para estudiar esta diapositiva

En signo y magnitud, un bit indica el signo y los otros $n-1$ guardan el valor absoluto. No leas el bit de signo como una potencia positiva que se añade a la magnitud.

Signo y magnitud (SM)

Ejemplo: **+28** en signo y magnitud

$28_{10} = 0011100_2 \rightarrow$ magnitud en siete bits.

Signo: 0 +



0	0	0	1	1	1	0	0
---	---	---	---	---	---	---	---

$$0 \mid 0011100_2 = 1C_{16}$$

El cero inicial es el signo; los siete bits restantes guardan el valor 28.

Para estudiar esta diapositiva

Para +28, el signo es 0 y los siete bits de magnitud codifican 28. La agrupación hexadecimal 1C es solo una abreviatura de esos ocho bits.

Signo y magnitud (SM)

Ejemplo: **-28** en signo y magnitud

$28_{10} = 0011100_2 \rightarrow$ magnitud en siete bits.

Signo: 1 -

1	0	0	1	1	1	0	0
---	---	---	---	---	---	---	---

$$1 \mid 0011100_2 = 9C_{16}$$

El uno inicial es el signo; los siete bits restantes guardan el valor 28.

Para estudiar esta diapositiva

Para -28 se conserva la magnitud y se cambia el signo a 1. El resultado 9C debe interpretarse como signo y magnitud; en otras convenciones tendría otro valor.

Signo y magnitud (SM)

Un bit de signo reduce la magnitud disponible

Sin signo: 8 bits $\rightarrow$ 0 ... 255



Con signo y magnitud: 7 bits $\rightarrow$ magnitud 0 ... 127

$$-(2^{n-1} - 1) \leq N \leq 2^{n-1} - 1$$

Con 8 bits: de -127 a +127. Hay 256 patrones, pero solo 255 valores distintos.

Para estudiar esta diapositiva

Con siete bits de magnitud se llega a 127 en ambos sentidos. Hay 256 patrones pero 255 valores distintos porque dos patrones representan el cero.

Signo y magnitud (SM)

Signo y magnitud tiene dos ceros

0	0	0	0	0	0	0	0	+0
1	0	0	0	0	0	0	0	-0

Dos patrones distintos representan el mismo valor: el cero.

Para estudiar esta diapositiva

+0 y -0 tienen signos diferentes y magnitud nula. Esta duplicación explica por qué signo y magnitud no aprovecha los patrones de la misma forma que CA2.

Cómo representamos los números enteros



A continuación: complemento a 1

Para estudiar esta diapositiva

En CA1, los positivos se mantienen como binario natural y los negativos se obtienen invirtiendo todos los bits. El ancho fijo es parte de la definición del código.

Complemento a 1 (CA1)

Positivos: igual que en binario natural.

+28

0	0	0	1	1	1	0	0
---	---	---	---	---	---	---	---

Para estudiar esta diapositiva

+28 se escribe 00011100 en ocho bits. Los ceros iniciales son importantes cuando se vaya a obtener su negativo mediante inversión.

Complemento a 1 (CA1)

Positivos: igual que en binario natural.

Negativos: invertimos todos los bits del positivo.

+28	0	0	0	1	1	1	0	0
	↓	↓	↓	↓	↓	↓	↓	↓
-28	1	1	1	0	0	0	1	1

El ancho es fijo: también se invierten los ceros iniciales.

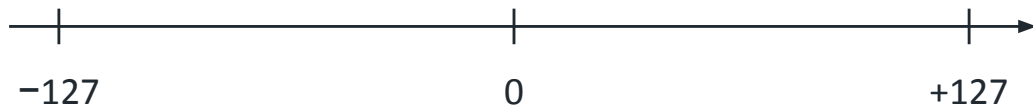
Para estudiar esta diapositiva

Invertir significa cambiar cada 0 por 1 y cada 1 por 0, incluidos los ceros iniciales. En CA1 no se añade una unidad después de invertir.

Complemento a 1 (CA1)

$$-(2^{n-1} - 1) \leq N \leq 2^{n-1} - 1$$

Valores representables



El número de bits limita el intervalo de valores que podemos representar.

Para estudiar esta diapositiva

CA1 tiene el mismo intervalo de valores que signo y magnitud, aunque sus códigos negativos son distintos. Con ocho bits, ambos extremos tienen magnitud 127.

Complemento a 1 (CA1)

CA1 tiene dos ceros. Al invertir todos los bits de +0 obtenemos -0.

0	0	0	0	0	0	0	0	+0
1	1	1	1	1	1	1	1	-0

Dos patrones distintos representan el mismo valor: el cero.

Para estudiar esta diapositiva

Al invertir 00000000 se obtiene 11111111. Ambos representan cero en CA1, por lo que también existe una duplicación de patrones para este valor.

Complemento a 1 (CA1) · Ejercicio 1

Representa -37 en complemento a 1 usando 8 bits.
Escribe el patrón binario y su abreviatura hexadecimal.

$-37_{10} \rightarrow \text{¿CA1?}$

Para estudiar esta diapositiva

Fija primero ocho posiciones y representa el valor absoluto 37. Después invierte las ocho cifras y agrupa de cuatro en cuatro para escribir hexadecimal.

Complemento a 1 (CA1) · Ejercicio 1

+37	0	0	1	0	0	1	0	1
Invertir	1	1	0	1	1	0	1	0

En CA1 basta con invertir todos los bits.

-37 en CA1: $1101\ 1010_2 = DA_{16}$.

Para estudiar esta diapositiva

$37 = 32 + 4 + 1$ produce 00100101 ; su inversión es 11011010 . El resultado DA_{16} representa -37 únicamente bajo la convención CA1 indicada.

Complemento a 1 (CA1) · Ejercicio 2

El patrón $D6_{16}$ codifica un entero en CA1 de 8 bits.
Obtén su valor decimal y explica cómo determinas el signo.

$D6_{16} \rightarrow \text{¿decimal?}$

Para estudiar esta diapositiva

$D6$ se despliega como 11010110. Decide primero el signo según CA1 y después aplica la operación inversa para recuperar la magnitud.

Complemento a 1 (CA1) · Ejercicio 2

D6 ₁₆	1	1	0	1	0	1	1	0
Invertir	0	0	1	0	1	0	0	1

En CA1 basta con invertir todos los bits.

El primer bit es 1: el número es negativo.

$00101001_2 = 32 + 8 + 1 = 41 \rightarrow$ valor decimal: -41.

Para estudiar esta diapositiva

La inversión da 00101001, que vale 41. Como el patrón original era negativo, el resultado es -41; no se suma uno al decodificar CA1.

Cómo representamos los números enteros



A continuación: complemento a 2

Para estudiar esta diapositiva

CA2 modifica la codificación negativa para disponer de un único cero y facilitar la suma. Su intervalo no es simétrico: contiene un valor negativo más que positivo.

Complemento a 2 (CA2)

Positivos y negativos

Positivos: igual que en binario natural.

Para obtener el código de un **negativo**:

Valor absoluto $\longrightarrow$ CA1 $\longrightarrow$ +1

Mantenemos el ancho fijo. El resultado es el patrón de complemento a 2.

Para estudiar esta diapositiva

Para codificar un negativo, parte del valor absoluto con el ancho fijado, obtén su CA1 y suma uno. El procedimiento se realiza sobre la palabra completa.

Complemento a 2 (CA2)

Ejemplo: $-28 \cdot \text{valor absoluto} \rightarrow \text{CA1} \rightarrow +1$

$ -28 $	0	0	0	1	1	1	0	0
	↓	↓	↓	↓	↓	↓	↓	↓
CA1	1	1	1	0	0	0	1	1
CA1 + 1	1	1	1	0	0	1	0	0

-28 en CA2: $1110\ 0100_2 = E4_{16}$. Ejemplo con 8 bits.

Para estudiar esta diapositiva

La secuencia para -28 es $00011100 \rightarrow 11100011 \rightarrow 11100100$. Separar estas fases ayuda a no omitir la suma final ni aplicarla al valor equivocado.

Complemento a 2 (CA2)

Un valor negativo más: de -128 a $+127$

$$-2^{n-1} \leq N \leq 2^{n-1} - 1$$

Valores representables



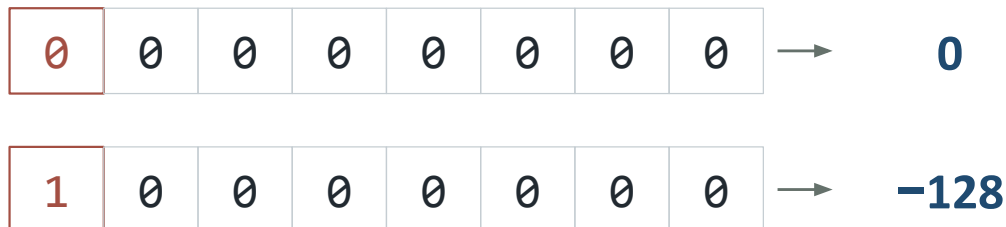
El número de bits limita el intervalo de valores que podemos representar.

Para estudiar esta diapositiva

En ocho bits, CA2 representa de -128 a 127 . Los 256 patrones corresponden ahora a 256 valores diferentes porque el cero tiene una sola representación.

Complemento a 2 (CA2)

Un único cero



Con 8 bits, +128 no cabe. Por eso -128 no tiene un opuesto representable en el mismo tipo.

Para estudiar esta diapositiva

El patrón 10000000 representa -128. Su opuesto matemático, +128, queda fuera del rango de ocho bits con signo; el ancho limita también qué negaciones son representables.

Complemento a 2 (CA2)

En decimal

$$7 + (-18) = -11$$

$$\begin{aligned} +7 &\rightarrow 00000111 \\ -18 &\rightarrow 11101110 \end{aligned}$$

En binario · CA2 de 8 bits

$$\begin{array}{r} 00000111 \\ + 11101110 \\ \hline 11110101 \end{array}$$

11110101_2 en CA2 representa -11 .

Mismo resultado: $7 - 18 = -11$. El resultado cabe en CA2 de 8 bits.

Para estudiar esta diapositiva

La suma de patrones produce 11110101 , cuyo valor CA2 es -11 . En este ejemplo no sale acarreo exterior y el resultado está dentro del intervalo, así que tampoco hay desbordamiento.

Complemento a 2 (CA2) · Ejercicio 1

Representa -37 en complemento a 2 usando 8 bits.
Escribe el patrón binario y su abreviatura hexadecimal.

$-37_{10} \rightarrow \text{¿CA2?}$

Para estudiar esta diapositiva

Representa 37 en ocho bits, invierte y añade uno. Comprueba que el resultado sea negativo según el bit más significativo y que no hayas cambiado el ancho.

Complemento a 2 (CA2) · Ejercicio 1

+37	0	0	1	0	0	1	0	1
CA1	1	1	0	1	1	0	1	0
Sumar 1	1	1	0	1	1	0	1	1

-37 en CA2: $1101\ 1011_2 = DB_{16}$.

Para estudiar esta diapositiva

La inversión de 00100101 da 11011010 y sumar uno da 11011011. Por eso -37 se abrevia DB_{16} en CA2, una unidad más que su código CA1.

Complemento a 2 (CA2) · Ejercicio 2

El patrón $D6_{16}$ codifica un entero en CA2 de 8 bits.
Obtén su valor decimal y explica cómo determinas el signo.

$D6_{16} \rightarrow \text{¿decimal?}$

Para estudiar esta diapositiva

El patrón hexadecimal no indica por sí mismo la convención. Aquí $D6$ se interpreta en CA2; debes recuperar la magnitud mediante inversión y suma de uno.

Complemento a 2 (CA2) · Ejercicio 2

D6 ₁₆	1	1	0	1	0	1	1	0
CA1	0	0	1	0	1	0	0	1
Sumar 1	0	0	1	0	1	0	1	0

El primer bit es 1: el número es negativo.

$00101010_2 = 32 + 8 + 2 = 42 \rightarrow$ valor decimal: -42.

Para estudiar esta diapositiva

Invertir 11010110 da 00101001 y añadir uno da 00101010, es decir, 42. El valor es -42, diferente del -41 obtenido al interpretar D6 en CA1.

Cómo representamos los números enteros



A continuación: representación sesgada

Para estudiar esta diapositiva

La representación sesgada desplaza todos los valores por una constante. El código almacenado se interpreta primero como entero sin signo y después se resta el sesgo.

Representación sesgada

Sumamos un desplazamiento al valor

$$\text{Código} = N + \text{Sesgo}$$

Elegimos $\text{Sesgo} = 2^{n-1}$. Con 8 bits: $\text{Sesgo} = 128$.

$$N + \text{Sesgo}$$

Guardamos el código en binario natural usando los n bits. Para recuperar N: código - Sesgo

Para estudiar esta diapositiva

El sesgo es un parámetro de la convención, no un valor universal. Aquí se elige 128 para ocho bits; más adelante el exponente de binary32 utilizará 127.

Representación sesgada

Ejemplo: +28 con sesgo 128

$$+28 + 128 = 156$$



$156_{10} = 1001\ 1100_2 = 9C_{16}$. Todos los bits forman el código; no hay bit de signo separado.

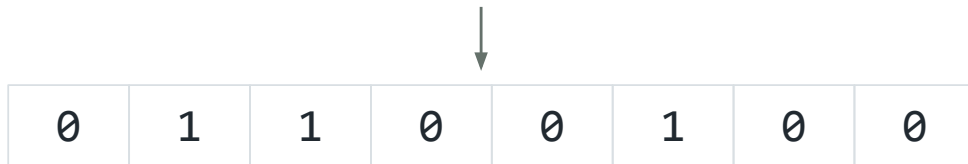
Para estudiar esta diapositiva

El valor +28 se desplaza a 156 y se escribe 10011100. No interpretes su primer 1 como signo negativo: todos los bits forman el código sin signo desplazado.

Representación sesgada

Ejemplo: -28 con sesgo 128

$$-28 + 128 = 100$$



$100_{10} = 0110\ 0100_2 = 64_{16}$. Al decodificar: $100 - 128 = -28$.

Para estudiar esta diapositiva

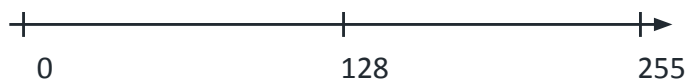
$-28 + 128 = 100$, que se codifica como 01100100. La comprobación inversa, $100 - 128 = -28$, permite validar la representación sin recurrir a reglas de complemento.

Representación sesgada

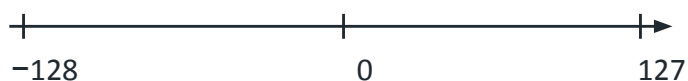
El sesgo desplaza el intervalo representable

$$-S \leq N \leq (2^n - 1) - S$$

Código



$N = \text{código} - 128$



Con $n = 8$ y $S = 128$: de -128 a $+127$. Mismo rango que CA2, distintos códigos.

Para estudiar esta diapositiva

El código sin signo recorre $0 \dots 2^n - 1$; restar S desplaza ambos extremos. Con $S=128$ coincide el rango de CA2, pero no la asignación de códigos.

Representación sesgada

El cero también se desplaza

$$N = 0 \rightarrow \text{código} = 0 + 128 = 128$$

1	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---

10000000: -128 en CA2 · 0 con sesgo 128

Un mismo patrón cambia de significado cuando cambia la convención.

Para estudiar esta diapositiva

El código 10000000 vale 128 sin signo y representa cero al restarle el sesgo 128.
Ese mismo patrón representa -128 en CA2: la convención es imprescindible.

Representación sesgada · Ejercicio

Representa -18 con 8 bits y sesgo 128.
Escribe el código en binario y en hexadecimal.
Comprueba que al decodificar recuperas -18 .

$-18_{10} \rightarrow \text{¿código?}$

Para estudiar esta diapositiva

Suma el sesgo antes de convertir a binario. Una comprobación completa consiste en leer después el código sin signo y restar 128 para recuperar -18 .

Representación sesgada · Ejercicio

$$\text{Código} = N + \text{Sesgo} = -18 + 128 = 110$$

$$110 = 64 + 32 + 8 + 4 + 2$$

$$\text{Comprobación: } 110 - 128 = -18$$

$$\mathbf{0110\ 1110_2 = 6E_{16}}$$

Para estudiar esta diapositiva

El código es 110 decimal, equivalente a 01101110_2 y $6E_{16}$. El primer bit a cero no significa que el valor original fuera positivo: el signo surge al deshacer el sesgo.

Cómo representamos los números enteros



A continuación: dígitos decimales codificados en binario

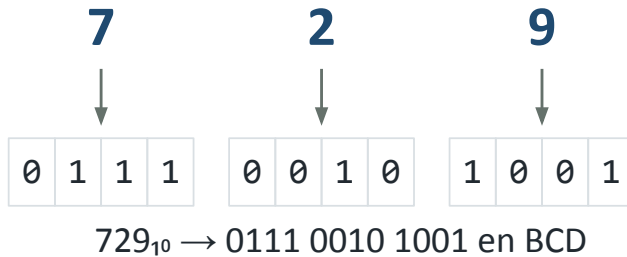
Para estudiar esta diapositiva

BCD cambia la unidad que se codifica: cada cifra decimal recibe cuatro bits. La cantidad completa no se transforma de una sola vez como en binario natural.

Datos enteros BCD

BCD = Binary-Coded Decimal (decimal codificado en binario).

Cada cifra decimal se representa con 4 bits.



Nº decimal	BCD
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Uso: calculadoras y relojes digitales; facilita procesar y mostrar números decimales cifra a cifra.

Para estudiar esta diapositiva

BCD significa Binary-Coded Decimal. En el código mostrado, los grupos 1010...1111 no representan cifras decimales; 729 usa los grupos de 7, 2 y 9, no el binario natural del entero 729.

Algunas cuestiones sobre la representación de enteros

Para estudiar esta diapositiva

Ahora se conectan códigos, rangos y operaciones. Distinguir un resultado matemático de los bits disponibles evitará confundir acarreo, desbordamiento e interpretación de tipos.

Desbordamiento (overflow): el resultado no cabe

El resultado matemático queda fuera del rango representable
(depende del tamaño de palabra: 8 bits, 16 bits, etc)

$$\text{CA2 con } n \text{ bits: } -2^{n-1} \leq N \leq 2^{n-1} - 1$$

Con 8 bits

Menor que -128

Fuera del rango

$$-128 \leq N \leq +127$$

Valores representables

Mayor que +127

Fuera del rango

Con 32 bits: de -2 147 483 648 a +2 147 483 647.

Para estudiar esta diapositiva

Hay desbordamiento cuando el resultado exacto no cabe en el rango elegido. No se decide mirando únicamente si aparece un bit adicional: también importa la interpretación con o sin signo.

Desbordamiento sin acarreo

$$127 + 1 = 128$$

128 queda fuera del rango
-128...127.

$$\begin{array}{r} 01111111 \\ + 00000001 \\ \hline 0\ 10000000 \end{array}$$

Acarreo exterior: 0
10000000 se interpreta como -128.

DESBORDAMIENTO: sí; 128 no es representable.

ACARREO: no; no sale ningún 1 de la palabra.

Para estudiar esta diapositiva

127+1 requiere representar 128, que no cabe en CA2 de ocho bits. El patrón resultante se leería como -128 y no hay acarreo exterior: desbordamiento y acarreo son distintos.

Acarreo sin desbordamiento

$$-1 + 1 = 0$$

El resultado 0 está dentro
del rango $-128 \dots 127$.

$$\begin{array}{r} 11111111 \\ + 00000001 \\ \hline 1\ 00000000 \end{array}$$

Acarreo exterior: 1

Se conservan los 8 bits: 00000000 $\rightarrow$ 0.

ACARREO: sale un bit de la palabra de 8 bits.
DESBORDAMIENTO: no; el resultado es representable.

Para estudiar esta diapositiva

$-1+1$ da cero, que sí cabe. Aparece un acarreo exterior al sumar los patrones, pero se conservan los ocho bits inferiores; la suma con signo no desborda.

C es un lenguaje tipado

Cada variable tiene un tipo: determina sus valores y operaciones.

Ejemplo habitual: int de 32 bits, short de 16 y char de 8.

Declaración	Bits	Representación	Rango
<code>int saldo = -18;</code>	32	Con signo · CA2	$-2^{31} \dots 2^{31} - 1$
<code>unsigned int total = 200u;</code>	32	Sin signo · binario	$0 \dots 2^{32} - 1$
<code>short temperatura = -300;</code>	16	Con signo · CA2	$-32\,768 \dots 32\,767$
<code>unsigned short n = 50000;</code>	16	Sin signo · binario	$0 \dots 65\,535$
<code>signed char cambio = -18;</code>	8	Con signo · CA2	$-128 \dots 127$
<code>unsigned char nivel = 200;</code>	8	Sin signo · binario	$0 \dots 255$

CA2 es obligatorio desde C23; antes se permitían también SM y CA1.

El tamaño depende de la implementación: int no siempre ocupa 32 bits.

43 / 47

Para estudiar esta diapositiva

Los tamaños de la tabla son los de la configuración ilustrada, no una promesa universal sobre int o short. Además, que una máquina use CA2 no convierte el desbordamiento de enteros con signo en una operación portable de C; no debe suponerse automáticamente un resultado circular. Referencia: borrador público de C23, apartados 6.2.6.2 y 6.5:

<https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3096.pdf> .

Los bits necesitan una interpretación



Sin signo: 249

CA2: -7

Mismo patrón, distinto valor. El tipo y la convención dan significado a los bits.

Para estudiar esta diapositiva

11111001 vale 249 sin signo y -7 en CA2. Si se almacena o transmite un patrón sin indicar cómo interpretarlo, se pierde información esencial sobre el valor que representa.

Calculamos $-14 + 7$ en CA2 de 8 bits

1. Codifica ambos operandos.
2. Suma sus patrones de ocho bits.
3. Interpreta el resultado y comprueba el desbordamiento.

$$-14 + 7$$

Para estudiar esta diapositiva

Resuelve primero las codificaciones y luego suma con ocho posiciones. La comprobación decimal $-14+7=-7$ permite comprobar el patrón y el posible desbordamiento por separado.

Primero obtenemos los dos operandos

+14

0	0	0	0	1	1	1	0
---	---	---	---	---	---	---	---

CA1

1	1	1	1	0	0	0	1
---	---	---	---	---	---	---	---

Sumar 1

1	1	1	1	0	0	1	0
---	---	---	---	---	---	---	---

-14 → 11110010. El otro operando es positivo: +7 → 00000111.

Para estudiar esta diapositiva

-14 se obtiene como 11110010 y +7 como 00000111. Solo el operando negativo necesita el procedimiento de valor absoluto, inversión y suma de uno.

Sumamos e interpretamos el resultado

$$\begin{array}{r} 11110010 \\ + 00000111 \\ \hline 11111001 \end{array}$$

El resultado empieza por 1:
es negativo en CA2.
Invertir y sumar 1 $\rightarrow$ 00000111.
Por tanto, vale -7 .

$-14 + 7 = -7$. Está dentro del rango $-128 \dots 127$: no hay desbordamiento.

Para estudiar esta diapositiva

El resultado 11111001 se decodifica como -7 . Sumar operandos de signos distintos no produce desbordamiento en CA2; en este caso tampoco aparece acarreo exterior.

Tema 2 · Nuestro recorrido

L2.0 Sistemas de numeración usuales en informática

L2.1 Nociones básicas sobre representación de la información

L2.2 Representación de textos

L2.3 Representación de sonidos

L2.4 Representación de imágenes

L2.5 Representación de vídeo

L2.6 Representación de números enteros

L2.7 Representación de números reales

Para estudiar esta diapositiva

La representación de reales reutiliza binario, sesgo y ancho fijo, pero introduce una mantisa y un exponente. El objetivo será equilibrar rango de magnitudes y precisión.

L2.7

Representación de números reales

01 Coma flotante 02 IEEE 754 03 Codificar
04 Decodificar 05 Límites

Para estudiar esta diapositiva

Primero se normaliza el número y después se rellenan campos de IEEE 754.
Codificar y decodificar son recorridos inversos que deben conservar el valor, salvo los efectos de redondeo.

Representación de números reales

Parte entera y fraccionaria

Un número real puede tener parte entera y parte fraccionaria. Por ejemplo: **53,25**.

Notación científica

mantisa × potencia de la base

$$53,25 = 53,25 \times 10^0 = 0,5325 \times 10^2 = 5,325 \times 10^1$$

Normalizar: dejar la primera cifra significativa en las unidades.

$$53,25 = 5,325 \times 10^1 \rightarrow 1 \leq M < 10$$

La coma se desplaza y el exponente compensa el cambio.

El valor se conserva. En binario, la mantisa normalizada cumple $1 \leq M < 2$.

Para estudiar esta diapositiva

La normalización elige una forma común entre escrituras equivalentes. Para una magnitud no nula, en base 10 queda una cifra no nula antes de la coma; en base 2 esa cifra necesariamente es 1.

Signo, mantisa y exponente

$$53,25 = (+1) \times 5,325 \times 10^1$$

Signo

S = +1

Mantisa

M = 5,325

Base b = 10

Exponente E = 1

$$N = S \times M \times b^E$$

- Notación exponencial; notación científica cuando está normalizada.
- En computación: coma flotante o punto flotante (floating point).
- Si E aumenta en 1, dividimos M por la base para conservar N.

Para estudiar esta diapositiva

S indica aquí el factor de signo, +1 o -1, mientras que s se usará después para el bit 0 o 1. Al desplazar la coma, el exponente compensa el cambio de mantisa.

Reales en binario

Fase 1 · De decimal fraccionario a binario

Parte entera: 53

$$53 = 32 + 16 + 4 + 1$$

$$53_{10} = 110101_2$$

Parte fraccionaria: 0,25

$$0,25 \times 2 = 0,5 \rightarrow 0$$

$$0,5 \times 2 = 1,0 \rightarrow 1$$

$$53,25_{10} = 110101,01_2$$

Comprobación: $32 + 16 + 4 + 1 + \frac{1}{4} = 53,25$.

Todavía no está normalizado: hay seis cifras antes de la coma.

Para estudiar esta diapositiva

La parte entera se obtiene por pesos y la fracción mediante multiplicaciones por dos. $0,25 = 1/4$ tiene representación binaria exacta, por lo que $53,25$ se convierte sin aproximación.

Reales en binario

Fase 2 · Normalizar y compensar

Estado inicial (sin normalizar)

$$110101,01_2 \times 2^0$$

6 cifras antes de la coma

Estado normalizado

$$1,1010101_2 \times 2^5$$

Exactamente 1 cifra no nula antes de la coma

Procedimiento de transformación:

Desplazamos la coma **5 posiciones a la izquierda** → Para conservar el valor, multiplicamos por **2^5**

Resultados clave:

- **Mantisa:** $M = 1,1010101_2$ | **Exponente:** $E = 5$ (Se cumple condición $1 \leq M < 2$)
- **Regla binaria:** En binario, todo número normalizado no nulo siempre empieza por **1**.

Para estudiar esta diapositiva

Desplazar cinco posiciones la coma transforma $110101,01_2$ en $1,1010101_2$. El factor 2^5 conserva la magnitud original; normalizar no cambia el número.

Reales en binario

Fase 3 · Codificar en IEEE 754 (en este ejemplo en 32 bits)

$53,25 = (+1) \times 1,1010101_2 \times 2^5$ · Precisión simple (32 bits)

1. Signo (s)

$s = 0 \rightarrow$ positivo

$s = 1 \rightarrow$ negativo

En este ejemplo: $s = 0$

1 bit

2. Exponente (e)

Sesgo = 127

$e = E + 127 = 132$

$= 10000100_2$

8 bits

3. Mantisa fraccionaria (Mf)

M es la mantisa normalizada

$M_f = M - 1$

$= 1,1010101_2 - 1 = 0,1010101_2$

23 bits

0

10000100

10101010000000000000000

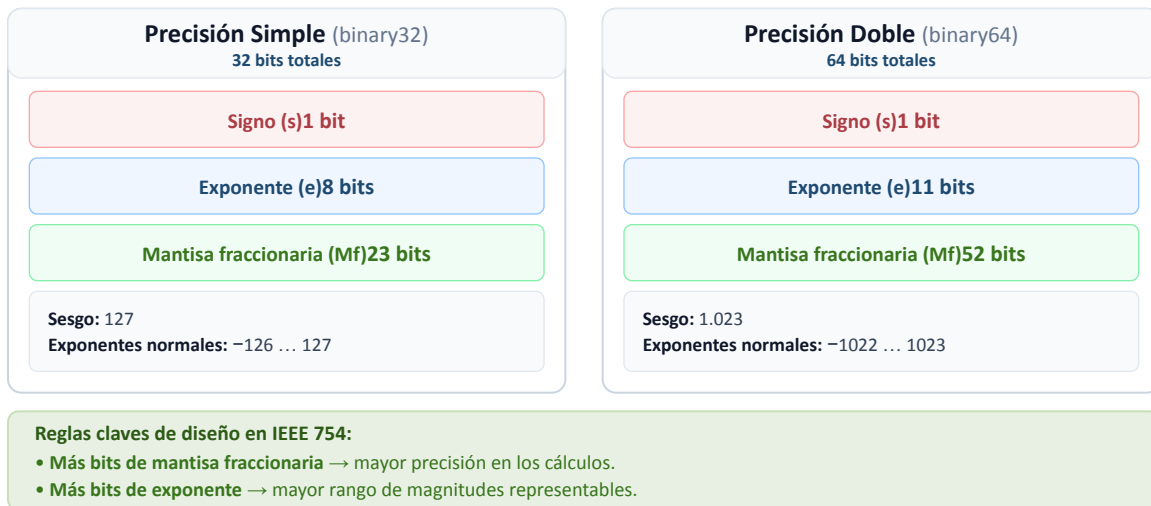
Detalle de almacenamiento de la mantisa:

- El 1 inicial es implícito en la representación normalizada ($1,1010101_2$).
- Guardamos los 23 bits tras la coma, completando con ceros a la derecha: 7 bits útiles + 16 ceros.

Para estudiar esta diapositiva

Este ejemplo utiliza 32 bits: uno de signo, ocho de exponente y 23 de mantisa fraccionaria. El exponente almacenado es $5+127=132$ y el 1 inicial normalizado se omite del campo fraccionario.

IEEE 754: precisión simple y doble



Para estudiar esta diapositiva

Binary32 y binary64 reparten de forma distinta sus bits entre exponente y fracción. Más bits de fracción aumentan la precisión; más bits de exponente amplían el rango de magnitudes.

Coma flotante en base 2

La norma IEEE 754

7 / 26

Para estudiar esta diapositiva

La norma define una interpretación compartida de los campos. La fórmula con un 1 implícito se aplica a números normales; ceros, subnormales, infinitos y NaN requieren reglas específicas.

Coma flotante en base 2 · La norma IEEE 754

$$N = S \times M \times 2^E$$

S · Signo

Valor: +1 o -1
Codificación: bit s
 $S = (-1)^s$

M · Mantisa

Rango: Normalizada
 $1 \leq M < 2$
 $M = 1 + M_f$

E · Exponente

Función: Potencia de dos
Ajuste con sesgo
 $E = e - \text{sesgo}$

Ejemplo práctico: $53,25 = (+1) \times 1,1010101_2 \times 2^5$

Números normales: $S = (-1)^s$; $M = 1 + M_f$; $E = e - \text{sesgo}$

Para estudiar esta diapositiva

En $N = S \times M \times 2^E$, distingue el exponente real E del código almacenado e . Para números normales se recupera E restando el sesgo, y M añadiendo el 1 implícito a la fracción.

Precisión simple · binary32



En C, habitualmente: `float x = 53.25f;` · Sesgo: 127.

Precisión: 24 bits = 1 implícito + 23 de mantisa fraccionaria.

Para estudiar esta diapositiva

Los 23 bits almacenados de fracción, más el 1 implícito, ofrecen 24 bits de precisión para números normales. La distribución de campos es independiente de cómo se ordenen los bytes en memoria.

Doble precisión · binary64



En C, habitualmente: `double x = 53.25;` · Sesgo: 1023.
Precisión: 53 bits = 1 implícito + 52 de mantisa fraccionaria.

Para estudiar esta diapositiva

Binary64 usa sesgo 1023 y 52 bits de fracción almacenada. No significa representar todos los reales exactamente: sigue siendo un conjunto finito de valores posibles.

Ejemplo: codificar 4_{10} en IEEE 754

PASO 1

Pasar a binario

Convertir el número decimal a su representación binaria pura.

PASO 2

Normalizar

Ajustar la coma para obtener la forma $1,xxx \times 2^E$ y determinar s , M , E .

PASO 3

Rellenar campos

Calcular el exponente con sesgo (e) y la mantisa fraccionaria (Mf).

PASO 1 · PASAR A BINARIO:

$$4_{10} = 100_2$$

Paso 2: normalizaremos y obtendremos **signo (s)**, **mantisa (M)** y **exponente (E)**.

Paso 3: rellenaremos los campos de la codificación: **e** (exponente sesgado) y **Mf** (mantisa fraccionaria).

Para estudiar esta diapositiva

El primer paso ya convierte 4 a 100_2 . Después se normaliza y se rellenan los campos; agrupar los 32 bits en hexadecimal será solo una forma adicional de escribir el resultado.

Ejemplo: codificar 4_{10} en IEEE 754

Paso 2 · Normalizar

VALOR EN BINARIO

$$100,0_2 \times 2^0$$

Coma al final



DESPLAZAMIENTO 1

$$10,0_2 \times 2^1$$

Mueve coma 1 pos. izq.



FORMA NORMALIZADA (1,xxx)

$$1,00_2 \times 2^2$$

✓ Normalización completa

PARÁMETROS EXTRAÍDOS DE LA FORMA $1,00_2 \times 2^2$: $S = 0$ (Signo positivo) $M = 1,00_2$ (Mantisa completa) $E = 2$ (Exponente real)

Cada movimiento de la coma a la izquierda incrementa el exponente para compensar y conservar el valor 4.

Para estudiar esta diapositiva

$100_2 = 1,0_2 \times 2^2$. La mantisa normalizada vale uno y el exponente real es dos; los movimientos de coma sirven para identificar ese exponente.

Ejemplo: codificar 4_{10} en IEEE 754

Paso 3 · Rellenar los campos

1. SIGNO (s) [1 bit]

$$s = 0$$

4 es positivo (+1)

2. EXPONENTE SESGADO (e) [8 bits]

$$e = 2 + 127 = 129$$

$$129_{10} = 10000001_2$$

3. MANTISA (Mf) [23 bits]

$$Mf = 0$$

$$M = 1,00_2 \rightarrow M - 1$$

DISTRIBUCIÓN FINAL EN LA PALABRA IEEE 754 (32 BITS):

s

0

1 bit

exponente e

10000001

8 bits

Mantisa fraccionaria

00000000000000000000000

23 bits

REGLA DE ALMACENAMIENTO DE LA MANTISA

$$M = 1,00_2 \rightarrow Mf = M - 1 = 0$$

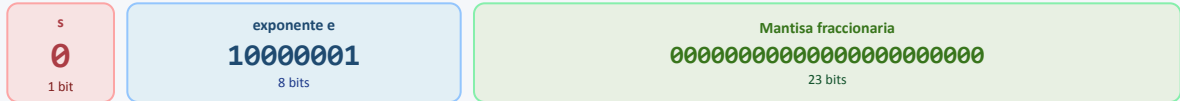
El 1 inicial de la mantisa normalizada ("bit implícito") es siempre 1 y no se almacena en el registro.

Para estudiar esta diapositiva

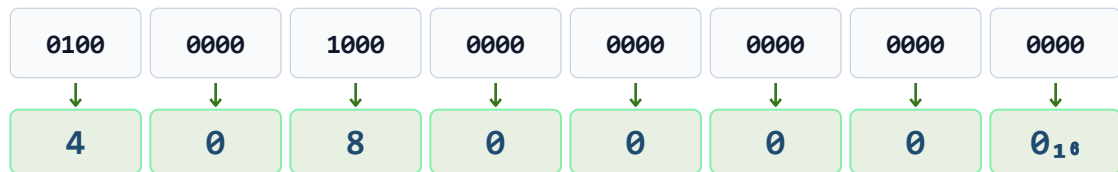
Para 4, $s=0$, $e=2+127=129$ y la mantisa fraccionaria es cero. No se almacena el 1 inicial de la mantisa normalizada.

Ejemplo: codificar 4_{10} en IEEE 754

Paso adicional · Agrupar en hexadecimal

 $s = 0$ (Signo) · $E = 2 \rightarrow e = 129$ (Exponente) · $Mf = 0$ (Mantisa)

AGRUPACIÓN EN BLOQUES DE 4 BITS (NIBBLES) Y TRADUCCIÓN A HEXADECIMAL:

 **$4_{10} \rightarrow 40800000_{16}$**

Para estudiar esta diapositiva

La agrupación hexadecimal se hace sobre la palabra completa, de cuatro en cuatro. El resultado para 4 en binary32 es 40800000; los límites de campos no tienen por qué coincidir con los de cada grupo hexadecimal.

Codificar $-11,0011_2$ · IEEE 754 de 32 bits

PASO 1**Dato en binario**

El número ya está en base 2. Separa su signo y su valor absoluto.

**PASO 2****Normalizar**

Desplaza la coma y compensa con una potencia de 2. Obtén s, M y E.

**PASO 3****Rellenar campos**

Aplica el sesgo 127 y calcula $M_f = M - 1$. Completa los 32 bits.

EJERCICIO · DATO YA EXPRESADO EN BINARIO

$-11,0011_2 \rightarrow$ ¿patrón IEEE 754?

Obtén el signo, el exponente sesgado y la mantisa fraccionaria.

Paso adicional: expresa el patrón de 32 bits en hexadecimal.

Para estudiar esta diapositiva

El dato $-11,0011_2$ ya está en binario y equivale a $-3,1875$. Se conserva el signo por separado; no se debe tratar la palabra IEEE 754 como un entero negativo en CA2.

Codificar $-11,0011_2$ · IEEE 754 de 32 bits**Paso 2 · Separar el signo y normalizar**

SIGNO DEL DATO

 $-11,0011_2$ Negativo $\rightarrow s = 1$ 

VALOR ABSOLUTO

 $11,0011_2 \times 2^0$

Desplaza la coma 1 posición



FORMA NORMALIZADA

 $1,10011_2 \times 2^1$ **$1 \leq M < 2$**

PARÁMETROS DE LA FORMA NORMALIZADA

 $s = 1$ $M = 1,10011_2$ $E = 1$

El signo se guarda aparte. No aplicamos CA2 a la mantisa.

Para estudiar esta diapositiva

La magnitud se normaliza como $1,10011_2 \times 2^1$. Por tanto, el signo es negativo, la mantisa normalizada es $1,10011_2$ y el exponente real es uno.

Codificar $-11,0011_2$ · IEEE 754 de 32 bits

Paso 3 · Rellenar los campos

1. SIGNO (s) [1 bit]

$$s = 1$$

El número es negativo.

2. EXPONENTE SESGADO [8 bits]

$$e = 1 + 127 = 128$$

$$128_{10} = 10000000_2$$

3. MANTISA FRACCIONARIA

$$Mf = 0,10011_2$$

$$Mf = 1,10011_2 - 1$$

DISTRIBUCIÓN FINAL EN LA PALABRA IEEE 754 (32 BITS):

s

1

1 bit

exponente e

10000000

8 bits

Mantisa fraccionaria

100110000000000000000000

23 bits

GUARDAMOS SOLO LOS BITS TRAS LA COMA

$$Mf = 0,10011_2 \rightarrow 100110000000000000000000$$

El 1 inicial es implícito. Añadimos 18 ceros a los 5 bits de la parte fraccionaria.

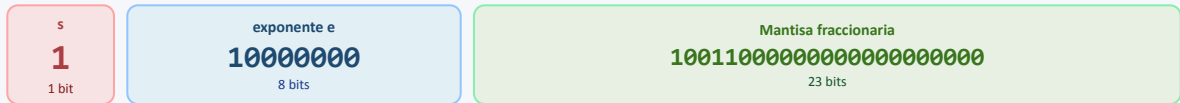
Para estudiar esta diapositiva

El bit de signo es 1 y el exponente almacenado es 128. La fracción 10011 se completa con ceros hasta 23 bits, sin almacenar el 1 implícito inicial.

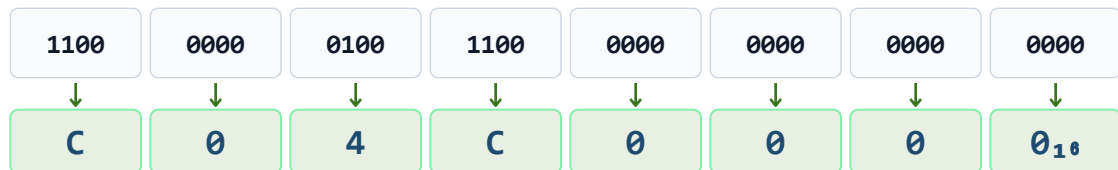
Codificar $-11,0011_2$ · IEEE 754 de 32 bits

Paso adicional · Agrupar en hexadecimal

$$s = 1 \quad \cdot \quad E = 1 \rightarrow e = 128 \quad \cdot \quad M_f = 0,10011_2$$



AGRUPACIÓN EN BLOQUES DE 4 BITS (NIBBLES) Y TRADUCCIÓN A HEXADECIMAL:



$$-11,0011_2 \rightarrow C04C0000_{16}$$

Para estudiar esta diapositiva

Al reunir los campos y agrupar en bloques de cuatro se obtiene C04C0000. El hexadecimal abrevia el patrón completo; no constituye una conversión del valor decimal como entero.

Decodificar $01F80000_{16}$ · IEEE 754 de 32 bits

PASO 1**Separar campos**

Convierte el hexadecimal a binario.
Identifica s, e y los bits de Mf.

**PASO 2****Recuperar S, E, M**

Interpreta el signo, resta el sesgo y recupera el 1 implícito.

**PASO 3****Calcular el valor**

Convierte la mantisa a decimal y calcula $N = S \times M \times 2^E$.

EJERCICIO · INTERPRETAR EL PATRÓN

$01F80000_{16} \rightarrow$ ¿valor decimal?

La palabra representa un número en IEEE 754 de precisión simple.
El hexadecimal abrevia sus 32 bits: no lo interpretes como un entero.

Para estudiar esta diapositiva

Primero despliega las ocho cifras hexadecimales en 32 bits. Después separa 1+8+23 y comprueba el tipo de exponente antes de aplicar la fórmula de un número normal.

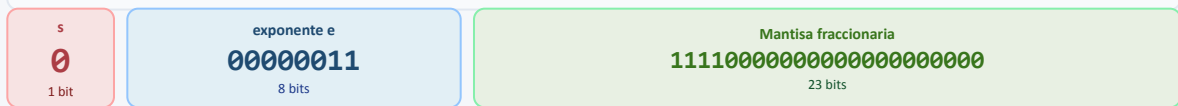
Decodificar 01F80000₁₆ · IEEE 754 de 32 bits

Paso 1 · Pasar a binario y separar los campos

CADA DÍGITO HEXADECIMAL EQUIVALE A CUATRO BITS:



Agrupar los 32 bits por campos: 1 de signo, 8 de exponente y 23 de mantisa.



Los límites de los campos no coinciden con los grupos de 4 bits.

Para estudiar esta diapositiva

En 01F80000, los campos son s=0, e=00000011 y fracción 1111000... . La separación de campos puede atravesar una cifra hexadecimal, por eso conviene desplegar todos los bits.

Decodificar 01F80000₁₆ · IEEE 754 de 32 bits

Paso 2 · Recuperar signo, exponente y mantisa

1. SIGNO

$$s = 0 \rightarrow S = +1$$

El número es positivo.

2. EXPONENTE REAL

$$E = 3 - 127 = -124$$

$$e = 00000011_2 = 3$$

3. MANTISA NORMALIZADA

$$M = 1,1111_2$$

$$M_f = 0,1111_2; M = 1 + M_f$$

PATRÓN DE PARTIDA: 01F80000₁₆

s

0

1 bit

exponente e

00000011

8 bits

Mantisa fraccionaria

11110000000000000000000

23 bits

PRIMERO COMPROBAMOS EL TIPO DE NÚMERO

$$1 \leq e = 3 \leq 254 \rightarrow \text{número normal}$$

Podemos recuperar el 1 implícito y usar $N = S \times M \times 2^E$.

Para estudiar esta diapositiva

El exponente almacenado vale 3, por lo que es un número normal y $E=3-127=-124$. La fracción $0,1111_2$ se completa con el 1 implícito para obtener $M=1,1111_2$.

Decodificar 01F80000₁₆ · IEEE 754 de 32 bits

Paso 3 · Calcular el valor decimal

SIGNO

$$S = +1$$

 $s = 0 \rightarrow$ positivo

POTENCIA DE DOS

$$2^E = 2^{-124}$$

 $E = -124$

MANTISA EN DECIMAL

$$M = 1,9375$$

 $1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16}$ EL MISMO PATRÓN DE PARTIDA: 01F80000₁₆

s

0

1 bit

exponente e

00000011

8 bits

Mantisa fraccionaria

11110000000000000000000

23 bits

RECONSTRUIMOS EL VALOR: $N = S \times M \times 2^E$

$$N = (+1) \times 1,9375 \times 2^{-124}$$

01F80000₁₆ representa aproximadamente $9,1100812 \times 10^{-38}$.

Para estudiar esta diapositiva

$1,1111_2 = 1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} = 1,9375$. Multiplicar por 2^{-124} produce aproximadamente $9,1100812 \times 10^{-38}$, con signo positivo.

Algunas cuestiones sobre la representación de reales

23 / 26

Para estudiar esta diapositiva

Un formato finito limita tanto magnitudes como precisión. Los casos especiales permiten interpretar patrones que no siguen exactamente la fórmula habitual de números normales.

Comprobación con un conversor IEEE 754

Valor Decimal

87₁₀

Flotante Hexadecimal

42AE0000₁₆

Decimal Decodificado

87₁₀

1. Normalización binaria

$$87 = 1010111_2 = 1,010111_2 \times 2^6$$

2. Desglose de campos (binary32)

Signo (s): 0 **Exponente (e):** 133 = 1000101₂**Mantisa (f):** 0101110000000000000000

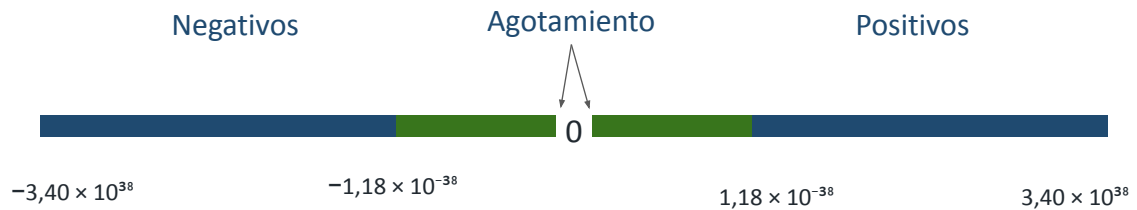
Comprobar interactivamente en:

https://www.binaryconvert.com/convert_float.html

Para estudiar esta diapositiva

Un conversor sirve como comprobación independiente después de calcular los campos. Para 87, E=6, e=133 y la palabra binary32 es 42AE0000; verifica que el conversor esté configurado en 32 bits.

Rango, desbordamiento y agotamiento



Números denormalizados cerca de cero

Mínimo positivo: $2^{-149} \approx 1,40 \times 10^{-45}$

Overflow: se supera la magnitud finita máxima.

Underflow: un resultado muy pequeño puede perder precisión o redondearse a cero.

Esquema no proporcional · En normales: $E = -126 \dots 127$.

25 / 26

Para estudiar esta diapositiva

Los valores representables no están igualmente espaciados en todo el eje. Los denormalizados prolongan la representación hacia cero con menor precisión relativa; la figura es esquemática y no está a escala.

Casos especiales en IEEE 754

Valor	Hexadecimal · IEEE 754 (32 bits)
0	00000000
inf	7F800000
-inf	FF800000
NaN	7FC00000 (un ejemplo; hay otros patrones)
1	3F800000

NaN: «Not a Number» (no es un número). El 1 se incluye como referencia.

26 / 26

Para estudiar esta diapositiva

Los códigos de la tabla describen palabras IEEE 754 de 32 bits. 00000000 corresponde a +0; también existe -0, con código 80000000. 7FC00000 es un ejemplo de NaN, no su único patrón. El valor 1 se incluye como referencia normal.

¡Gracias por vuestra atención!

Para estudiar esta diapositiva

Para repasar, elige un patrón y explica qué información adicional necesitas para interpretarlo. Comprueba también que sabes distinguir bits y bytes, tamaño y tasa, valor y código, y representación exacta y aproximada.